FPT UNIVERSITY

CAPSTONE PROJECT REPORT

Report 6 – Software User Guides

Frevia – AI-Powered Freelance Marketplace & Community Platform

Author / In Charge:	Nguyen Tien Thuan (CE181024)
Project Name:	Frevia Platform
Academic Year:	2024 – 2025
Supervision:	FPT University

– March 2025 –

Table of Contents

I. Record of Changes **3**

II. Release Package & User Guides **4**

1 Deliverable Package **4**

2 Installation Guides **6**

2.1 System Requirements 6

2.1.1 Hardware Requirements 6

2.1.2 Software Prerequisites 6

2.2 Software Prerequisites & Download Instructions 7

2.2.1 Step 1: Download and Install Git SCM 7

2.2.2 Step 2: Download and Install Node.js LTS and pnpm 8

2.2.3 Step 3: Download and Install Docker Desktop 9

2.2.4 Step 4: Verify Installed Runtime Environments 10

2.3 Third-Party Services Configuration & Environment Variables 10

2.4 Monorepo Setup, Dependencies & Database Initialization 11

2.4.1 Step 1: Clone Repository and Install Dependencies 11

2.4.2 Step 2: Configure Environment Files 12

2.4.3 Step 3: Database Schema Migration and Initial Seeding 13

2.5 Service Port Mapping 13

2.6 System Startup Options 14

2.6.1 Option 1: Automated Deployment via Docker Compose (Recommended) . 14

2.6.2 Option 2: Local Development Workflow 15

2.7 Post-Installation System Verification 16

2.7.1 1. Frontend Web Application Verification 16

2.7.2 2. Backend RESTful API & Health Check Verification 17

2.7.3 3. Pre-Configured Test Accounts 17

3 User Manual **18**

3.1 Overview of the Application & System Actors 18

3.2 Module 2: Authentication & Session Management 19

3.2.1 M2.01 – New User Registration & Email OTP Verification (UC-01) 19

3.2.2 M2.02 – User Authentication via Email & Password (UC-02) 21

3.2.3 M2.03 – Single Sign-On via Google OAuth 2.0 (UC-03) 23
3.2.4 M2.04 – Password Recovery & Reset via Email OTP (UC-04) 24
3.2.5 M2.05 – Active Login Session Monitoring & Device Audit (UC-05.01) . . 26
3.2.6 M2.06 – Remote Session Revocation & Security Enforcement (UC-05.02) . 27
3.2.7 M2.07 – Secure User Sign-Out & Session Invalidation (UC-05.03) 28
3.3 Module 14: Administrator System, RBAC & User Management 29
3.3.1 M14.01 – Administrator Central Operations Dashboard (`/admin`) 29
3.3.2 M14.02 – Administrator Profile, Identity & System Privileges (`/admin/users/1`) 31
3.3.3 M14.03 – System Roles & Custom Roles Management (UC-06.01 to UC-06.05) 32
3.3.4 M14.04 – Dynamic Role-Permission Assignment Matrix (UC-06.06) . . . 34
3.3.5 M14.05 – System API Permission Catalog & Multi-Criteria Filtering (UC-15.01 to UC-15.04) 35
3.3.6 M14.06 – User Directory Administration & Direct Account Provisioning (UC-07.01 to UC-07.03) 37
3.3.7 M14.07 – User Account Status Management & Suspension/Reactivation (UC-07.04 to UC-07.05) 39
3.3.8 M14.08 – Comprehensive User Multi-Profile Management (UC-07.06 to UC-07.07) 41
3.4 Module 15: Administrator Master Data Catalog Management 42
3.4.1 M15.01 – Skills Catalog Management with Soft Delete & Restore (UC-11.01 to UC-11.06) 42
3.4.2 M15.02 – Job Categories & Industry Classification Management (UC-12.01 to UC-12.06) 44
3.4.3 M15.03 – Forum Discussion Categories Management (UC-13.01 to UC-13.06) 46

I. Record of Changes

Date	A*, M, D	In charge	Change Description
2025-03-05	A	Nguyen Tien Thuan	Initial draft of Report 6 template structure, document metadata, and environment requirements.
2025-03-08	A	Nguyen Tien Thuan	Captured 28 verified screenshots across Part A (Installation) and Part B (Modules 2, 14, 15).
2025-03-08	M	Nguyen Tien Thuan	Added red bounding boxes and standardized numeric callout badges to all captured screenshots.
2025-03-09	M	Nguyen Tien Thuan	Translated entire document content, installation instructions, and user manual to formal English.
2025-03-09	A	Nguyen Tien Thuan	Finalized PDF compilation, verified cross-references, table of contents, and formatting compliance.

**A - Added M - Modified D - Deleted*

II. Release Package & User Guides

1. Deliverable Package

The official release package (*Release Package v1.0*) of the **Frevia – AI-Powered Freelance Marketplace & Community Platform** project encompasses the complete monorepo source codebase, database migration schemas, automated seed scripts, containerization specifications, and the comprehensive suite of engineering reports from Report 1 through Report 7. Table 1 outlines the deliverable inventory:

No.	Deliverable Item	Description & Storage Path	Version / Format
1	Project Schedule & Tracking	Comprehensive project roadmap, milestone tracking, resource allocations, and sprint burndown schedules.	`v1.0` (`.xlsx`)
2	Product Backlog	Prioritized list of User Stories, Use Cases, acceptance criteria, and story point estimations across all modules.	`v1.0` (`.xlsx`)
3	Source Codes (Monorepo)	Full application source code managed via **Turborepo** and **pnpm workspaces**: Backend REST/WebSocket API (`apps/api` – NestJS 11, Prisma 7), Frontend Web App (`apps/web` – Next.js 16, React 19, Tailwind CSS v4), and shared packages (`packages/shared-types`, `packages/ui`).	`v1.0.0` (`Git`)
4	Database Scripts & Seed Data	PostgreSQL relational schema containing 39 entity models (`apps/api/prisma/schema.prisma`), 14 sequential migration scripts, and automated database seeders (`apps/api/src/seed-datas/`).	`v1.0` (`.prisma/.ts`)
5	Technical Reports (R1–R7)	Complete formal documentation suite complying with FPT University Capstone Project standards: Report 1 (Introduction), Report 2 (Project Management Plan), Report 3 (SRS), Report 4 (SDS), Report 5 (Testing Document), Report 6 (User Guides), and Report 7 (Final Report).	`v1.0` (`.pdf/.tex`)

No.	Deliverable Item	Description & Storage Path	Version / Format
6	Unit & Integration Test Cases	Comprehensive unit test specification covering 121 API and service functions (`Report5_Unit Test.xls`) and automated Jest test suites (`apps/api/src/**/*.spec.ts`).	`v1.0 (.xls/.ts)`
7	Defects & Issues Log	Defect tracking ledger logging bug discoveries, severity categorizations, reproduction steps, root cause analyses, and resolution statuses.	`v1.0 (.xlsx)`
8	Docker & Orchestration Configs	Multi-service container orchestration scripts: `docker-compose.yml`, multi-stage container build specifications (`apps/api/Dockerfile`, `apps/web/Dockerfile`), and environment templates (`.env.example`).	`v1.0 (.yml)`
9	Presentation Slides & Demo	Formal thesis defense slide deck and live demonstration script showcasing key system workflows.	`v1.0 (.pptx)`

Monorepo Directory Structure

The system is architected as an enterprise-grade **Turborepo Monorepo**, guaranteeing strict type-safety and shared Zod validation schemas across both client-side and server-side runtimes:

```
frevia-clone/
|-- apps/
|   |-- api/                  # Backend RESTful API & WebSocket (NestJS 11 + Prisma 7)
|   |   |-- prisma/           # schema.prisma (39 entity models) & SQL migrations
|   |   |-- src/modules/      # 25+ business domain modules (auth, users, roles, jobs...)
|   |   |-- src/seed-datas/   # Seeders for RBAC permissions, roles, and test users
|   |   `-- Dockerfile        # Multi-stage production container build for NestJS API
|   `-- web/                  # Frontend Web App (Next.js 16 App Router + Tailwind v4)
|       |-- app/              # Application routing: (auth), (main), admin, sessions...
|       |-- apiRequests/      # Typed HTTP client layer & Next.js Server Actions
|       `-- Dockerfile        # Multi-stage production container build for Next.js Web
|-- packages/
|   |-- shared-types/         # Shared Zod schemas, DTOs, Enums across API & Web
|   |-- ui/                   # Shared UI component library (Radix UI / Shadcn UI)
|   |-- eslint-config/        # Standardized code quality and linting configurations
|   `-- typescript-config/    # Monorepo-wide TypeScript compilation settings
|-- docs/                     # Capstone documentation suite (Reports 1 to 7)
|-- docker-compose.yml        # Orchestration for 5 containers: postgres, redis, pgadmin,
    api, web
|-- package.json              # Monorepo script coordinator & workspace definitions
`-- turbo.json                # Turborepo build, lint, and test caching pipeline
```

2. Installation Guides

2.1 System Requirements

2.1.1 Hardware Requirements

To ensure optimal execution and simultaneous operation of the database, memory cache, RESTful API server, and server-rendered web application, the host system must meet or exceed the following hardware specifications:

Component	Minimum Specification	Recommended Specification
CPU	4 cores (Intel Core i5 Gen 8 / AMD Ryzen 5 / Apple M1)	8 cores or higher (Intel Core i7 / AMD Ryzen 7 / Apple M2 Pro)
RAM	8 GB (minimum 3.5 GB free for Docker & Node.js)	16 GB or higher
Storage	10 GB available SSD storage	20 GB or higher available NVMe SSD storage
Network	Broadband internet connection ($\geq$ 10 Mbps)	High-speed internet connection ($\geq$ 50 Mbps)

2.1.2 Software Prerequisites

The following software environments, compilers, and container engines must be installed on the deployment machine:

Software Environment	Required Version	Role & Functional Scope
Operating System	Windows 10/11 (64-bit), macOS 13+, or Ubuntu 22.04 LTS	Host OS supporting Docker virtualization with WSL2 or native Linux containers.
Docker Desktop	Version 4.30.0 or higher	Container engine providing virtualization for PostgreSQL, Redis, pgAdmin, API, and Web.
Node.js Runtime	Version 20.x or 22.x (LTS)	JavaScript/TypeScript V8 runtime environment powering both NestJS and Next.js.
pnpm Package Manager	Version 9.x or higher	High-performance, disk-efficient package manager coordinating the monorepo workspace.
Git SCM	Version 2.40.0 or higher	Distributed version control system used to clone repositories and manage revisions.

2.2 Software Prerequisites & Download Instructions

Deploying the **Frevia** platform requires three core development runtimes and orchestration utilities:

2.2.1 Step 1: Download and Install Git SCM

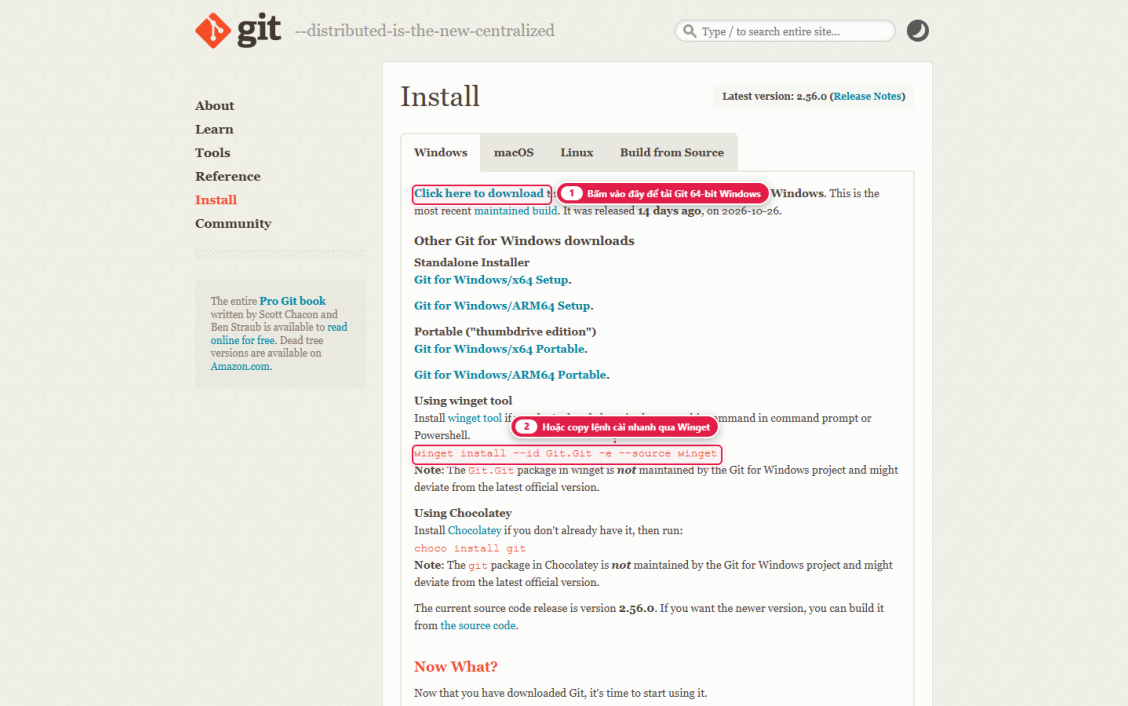


Figure 2.1: Downloading the Git SCM installer for 64-bit Windows from the official website

- **[1]** Navigate to the official Git download portal at `https://git-scm.com/downloads/win`.
- **[2]** Click **Click here to download** to retrieve the latest 64-bit standalone installer for Windows (`Git-2.xx.x-64-bit.exe`).
- Execute the installer, ensuring **Git Bash** and **Git Credential Manager** options are enabled during setup.

2.2.2 Step 2: Download and Install Node.js LTS and pnpm

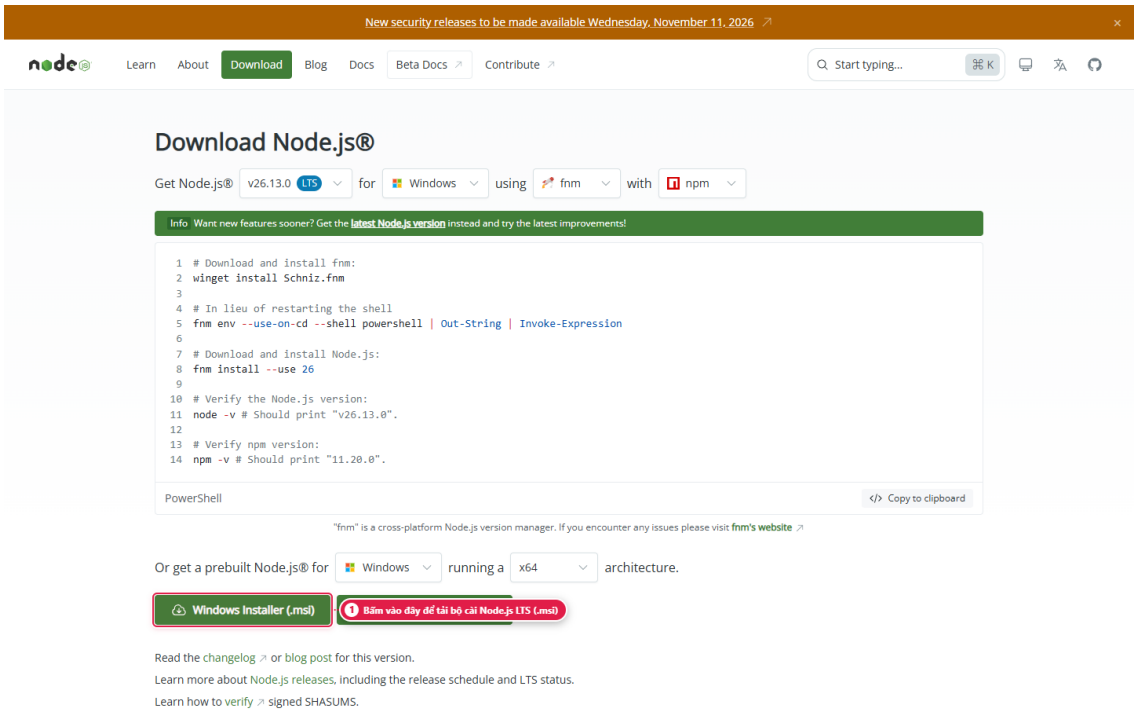


Figure 2.2: Downloading the Node.js Long Term Support (LTS) installer from the official website

- **[1]** Navigate to the official Node.js download portal at `https://nodejs.org/en/download`.
- **[2]** Select the stable **Node.js LTS (v22.x or v20.x)** distribution and download the 64-bit Windows Installer (`.msi`).
- After completing the Node.js installation, open a terminal with administrator privileges and run:

```
# Enable Corepack and install the latest pnpm package manager
corepack enable
corepack prepare pnpm@latest --activate
```

2.2.3 Step 3: Download and Install Docker Desktop

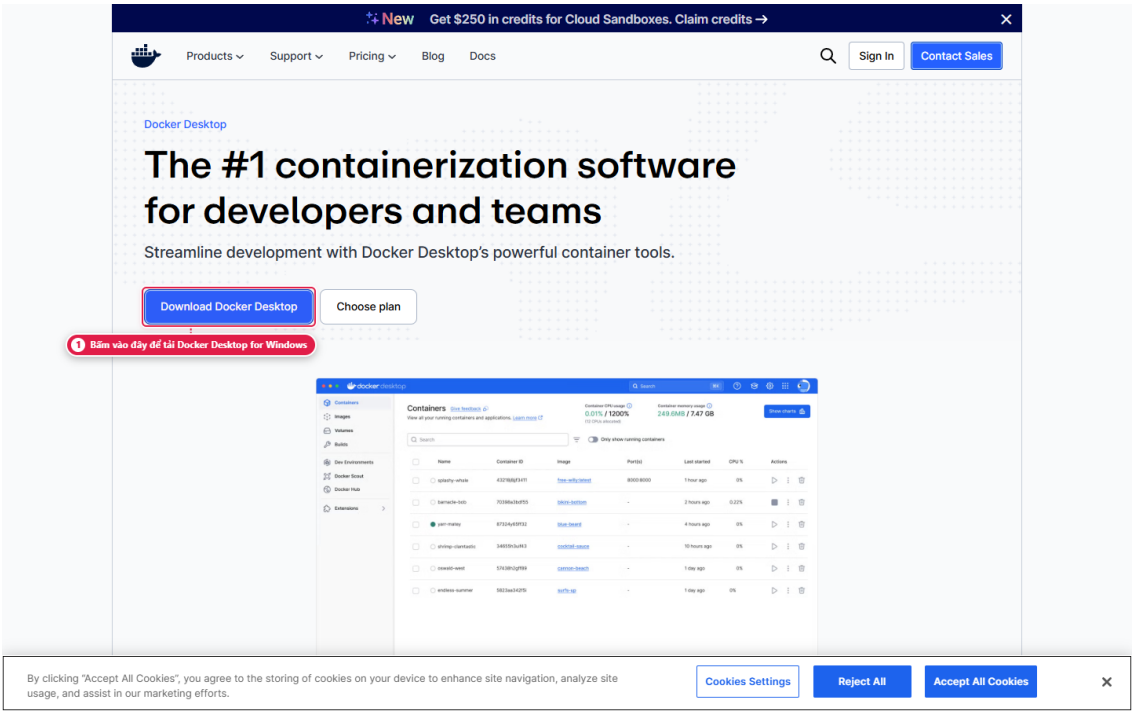


Figure 2.3: Downloading the Docker Desktop installer for Windows with WSL2 support

- **[1]** Navigate to the official Docker Desktop download portal at `https://www.docker.com/products/docker-desktop/`.
- **[2]** Click **Download for Windows** to retrieve `Docker Desktop Installer.exe`.
- Run the installer and ensure the option **Use WSL 2 instead of Hyper-V (recommended)** is checked. Restart the computer if prompted.

2.2.4 Step 4: Verify Installed Runtime Environments

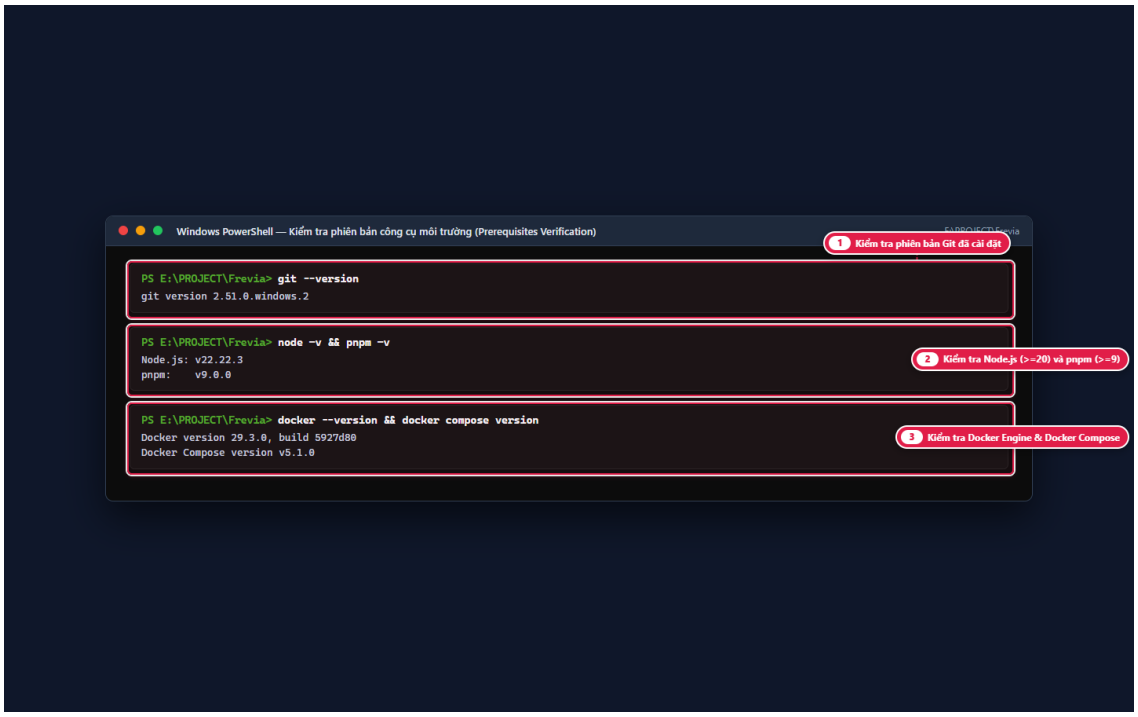


Figure 2.4: Verifying installed version signatures of Git, Node.js, pnpm, and Docker via PowerShell

Open Windows PowerShell or a Unix terminal and execute the verification commands shown above:

- **[1]** Execute `git -version` → confirms Git version (e.g., `git version 2.47.1.windows.1`).
- **[2]** Execute `node -v` → confirms Node.js LTS version (e.g., `v22.12.0`).
- **[3]** Execute `pnpm -v` → confirms pnpm version (e.g., `9.15.0`).
- **[4]** Execute `docker -v` and `docker compose version` → confirms Docker Desktop engine (e.g., `Docker version 27.4.0, build bde2b89`) and Docker Compose plugin (e.g., `Docker Compose version v2.31.0`).

2.3 Third-Party Services Configuration & Environment Variables

The **Frevia** platform integrates several external cloud APIs to support authentication, asset storage, real-time communications, secure escrow payments, and AI capabilities:

Service Provider	Environment Variables	Role & Functional Scope
Google Identity	`GOOGLE_CLIENT_ID` `GOOGLE_CLIENT_SECRET`	Google OAuth 2.0 Single Sign-On (SSO) integration.

Service Provider	Environment Variables	Role & Functional Scope
Gmail SMTP	`MAIL_HOST`, `MAIL_PORT` `MAIL_USER`, `MAIL_PASS`	Transmission of 6-digit email OTPs for registration, password recovery, and notifications.
Cloudinary CDN	`CLOUDINARY_CLOUD_NAME` `CLOUDINARY_API_KEY` `CLOUDINARY_API_SECRET`	High-performance cloud hosting and image optimization for avatars, portfolios, and job attachments.
Stripe Connect	`STRIPE_SECRET_KEY` `STRIPE_WEBHOOK_SECRET`	Milestone escrow payment processing, customer checkouts, and automated freelancer payouts.
Google Gemini AI	`GEMINI_API_KEY`	Generative AI engine powering CV extraction, automated skill matching, and smart job analysis.

2.4 Monorepo Setup, Dependencies & Database Initialization

2.4.1 Step 1: Clone Repository and Install Dependencies



Figure 2.5: Cloning repository and executing pnpm install across all monorepo packages

- **[1]** Run `git clone <repository_url>` to clone the project to the local workspace.
- **[2]** Run `pnpm install` at the workspace root directory. The monorepo resolver links all shared packages (`packages/shared-types`, `packages/ui`) and resolves third-party dependencies efficiently.

2.4.2 Step 2: Configure Environment Files

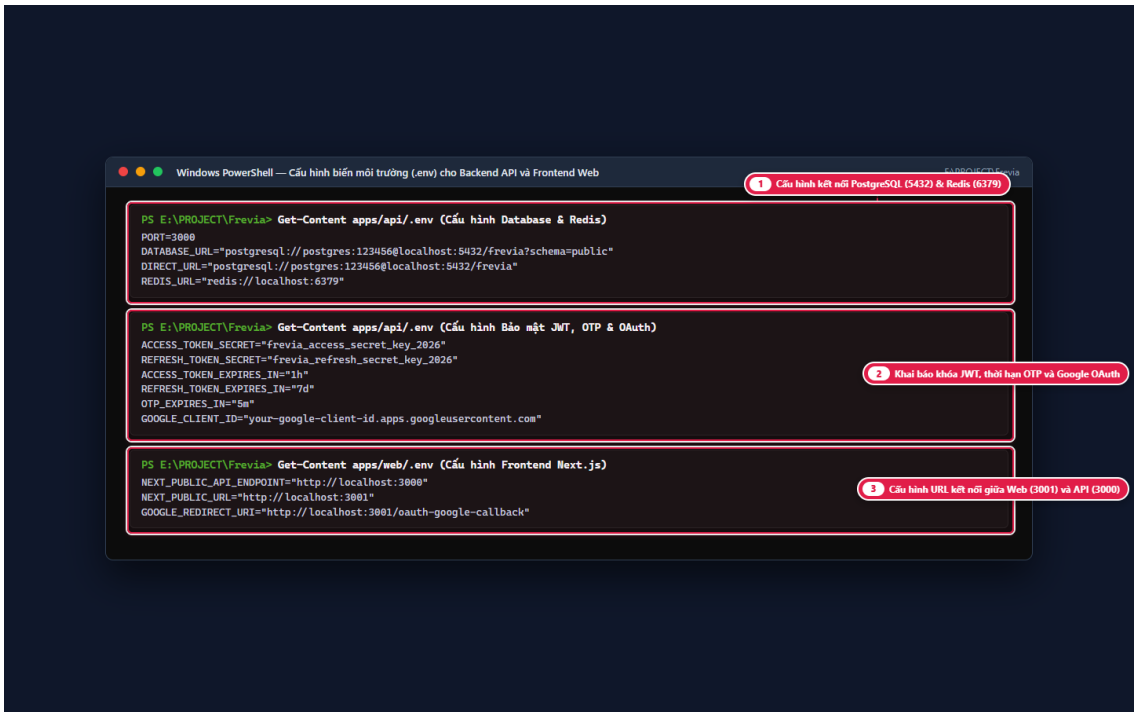


Figure 2.6: Verifying environment configuration files for API (`apps/api/.env`) and Web (`apps/web/.env`)

- **[1]** Navigate to `apps/api` and duplicate `.env.example` to create `.env`. Verify database credentials, Redis configuration, JWT secrets, and external API keys:

```
DATABASE_URL="postgresql://frevia_user:frevia_password@localhost:5432/frevia?schema=
    public"
REDIS_HOST="localhost"
REDIS_PORT=6379
JWT_SECRET="frevia_jwt_secret_production_key"
PORT=3000
```

- **[2]** Navigate to `apps/web` and create `.env`. Configure public and server-side endpoints:

```
NEXT_PUBLIC_API_ENDPOINT="http://localhost:3000"
NEXT_PUBLIC_URL="http://localhost:3001"
API_ENDPOINT="http://localhost:3000"
```

2.4.3 Step 3: Database Schema Migration and Initial Seeding

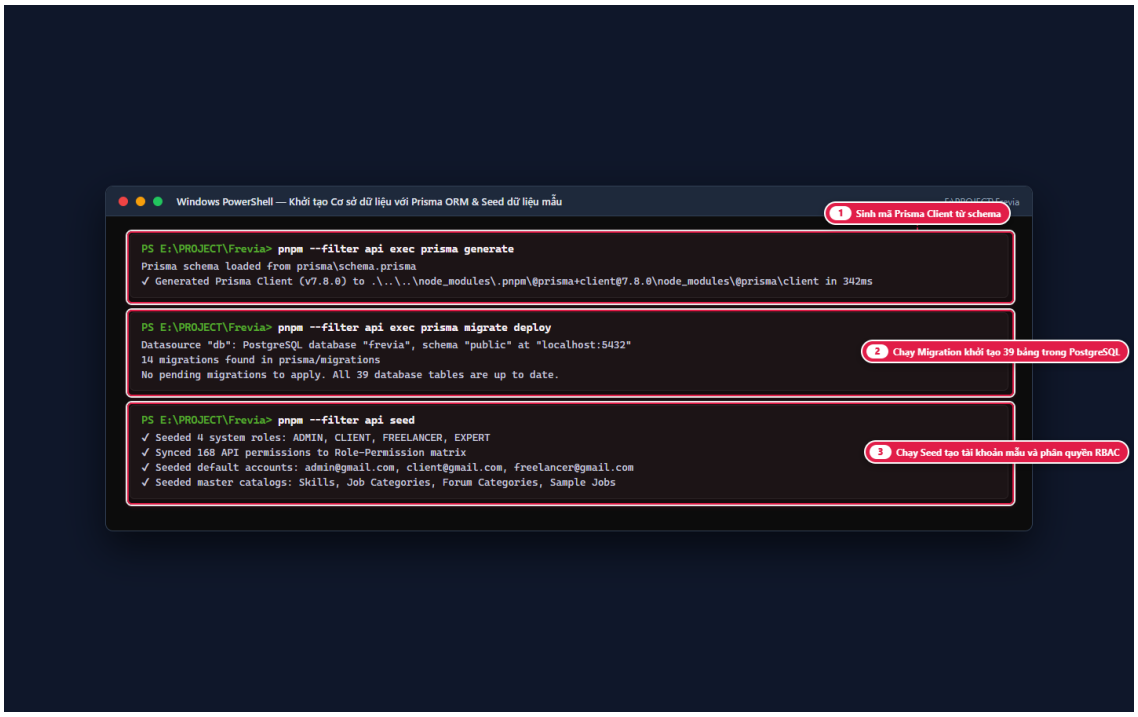


Figure 2.7: Generating Prisma Client, executing database migrations, and executing seed scripts

Open the terminal in `apps/api` (or pass filter flags from the monorepo root) and execute the database lifecycle commands:

- **[1]** Execute `pnpm -filter @frevia/api prisma generate` to compile the Prisma TypeScript Client.
- **[2]** Execute `pnpm -filter @frevia/api prisma migrate deploy` to apply all 14 database migrations to the PostgreSQL instance.
- **[3]** Execute `pnpm -filter @frevia/api seed` to populate default RBAC roles, all 275 granular system permissions, pre-configured system admin accounts, master skill catalogs, and sample category dictionaries.

2.5 Service Port Mapping

When operating the complete system on a local host or cloud virtual machine, the **Frevia** architecture utilizes the following default network ports. The system administrator or developer must verify that these ports are unoccupied prior to starting the containers:

Service Name	Container Name	Port	Endpoint URL & Functional Role
Frontend Web	`frevia-web`	3001	`http://localhost:3001` – Next.js 16 Client Portal and Admin Management Console.
Backend API	`frevia-api`	3000	`http://localhost:3000` – NestJS 11 RESTful API endpoints and WebSocket gateway.
PostgreSQL DB	`frevia-postgres`	5432	`localhost:5432` – Primary relational database storage (Database: `frevia`).
Redis Server	`frevia-redis`	6379	`localhost:6379` – In-memory caching engine, OTP storage, and session broker.
pgAdmin 4	`frevia-pgadmin`	5050	`http://localhost:5050` – Web-based database management and visualization console.

2.6 System Startup Options

The platform supports **two distinct execution workflows** tailored for production-like evaluation and daily development:

2.6.1 Option 1: Automated Deployment via Docker Compose (Recommended)

This workflow packages and orchestrates all 5 system services (`postgres`, `redis`, `pgadmin`, `api`, `web`) within an isolated internal bridge network (`frevia-network`) using a single terminal command:

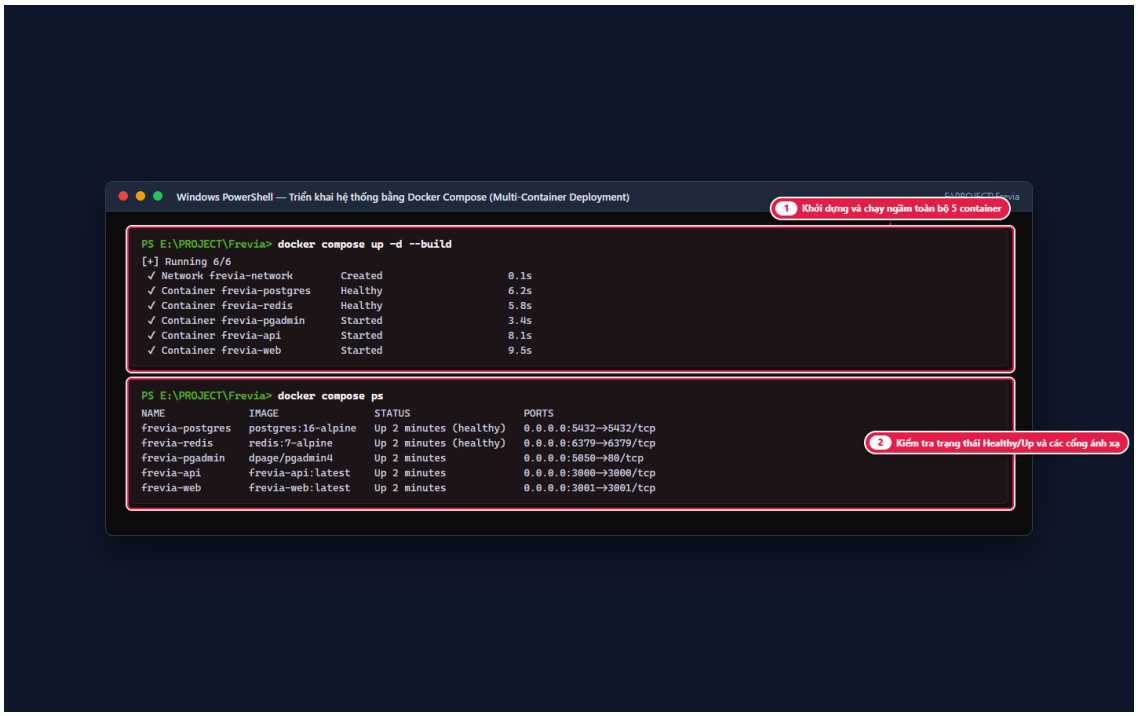


Figure 2.8: Building and deploying all 5 system containers using Docker Compose

- **[1]** In the root directory of the repository, execute:

```
docker compose up -d --build
```

  Docker builds production images for the API and Web applications, initializes persistent volumes, and launches all containers in detached mode.

- **[2]** Run `docker compose ps` to verify container execution states: confirm that the database and cache report `Up (healthy)` status and the port mappings align with the architecture specification.

- Routine monitoring and shutdown commands:

```
# Tail live logs for the API and Web services
docker compose logs -f api web

# Gracefully terminate all containers while retaining database volume data
docker compose down
```

2.6.2 Option 2: Local Development Workflow

For active development with hot-module reloading and interactive debugging:

1. Start the backend infrastructure services via Docker:

```
docker compose up -d postgres redis pgadmin
```

2. Launch the NestJS backend and Next.js frontend concurrently across the monorepo:

```
pnpm dev
```

   Turborepo coordinates concurrent processes: the NestJS API compiles on port `3000` and Next.js compiles on port `3001` with instantaneous hot reloading.

2.7 Post-Installation System Verification

Following deployment, evaluators should verify system availability across both client-side and server-side tiers:

2.7.1 1. Frontend Web Application Verification

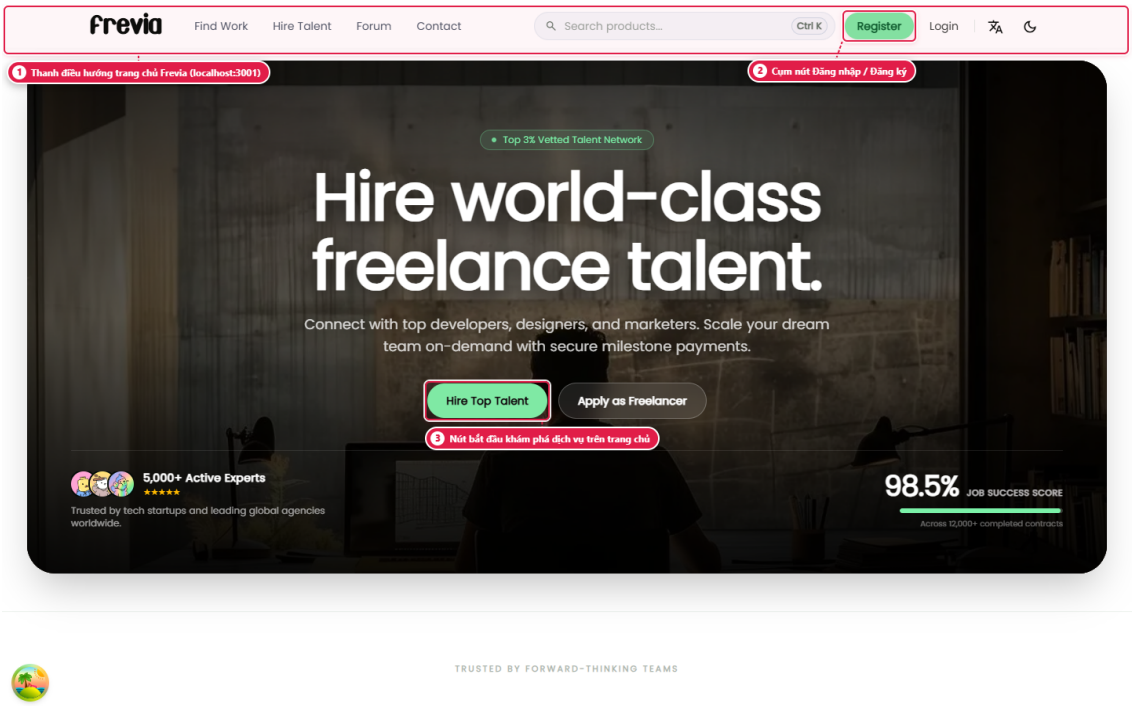


Figure 2.9: Verifying the Next.js web application running at http://localhost:3001

- **[1]** Open a modern web browser and navigate to `http://localhost:3001`. Confirm that the header renders navigation links (*Browse Jobs*, *Freelancers*, *Community*) alongside **Sign in** and **Sign up** buttons.
- **[2]** Verify that the hero banner renders correctly with the AI search interface, featured skills pills, and call-to-action buttons.

2.7.2 2. Backend RESTful API & Health Check Verification

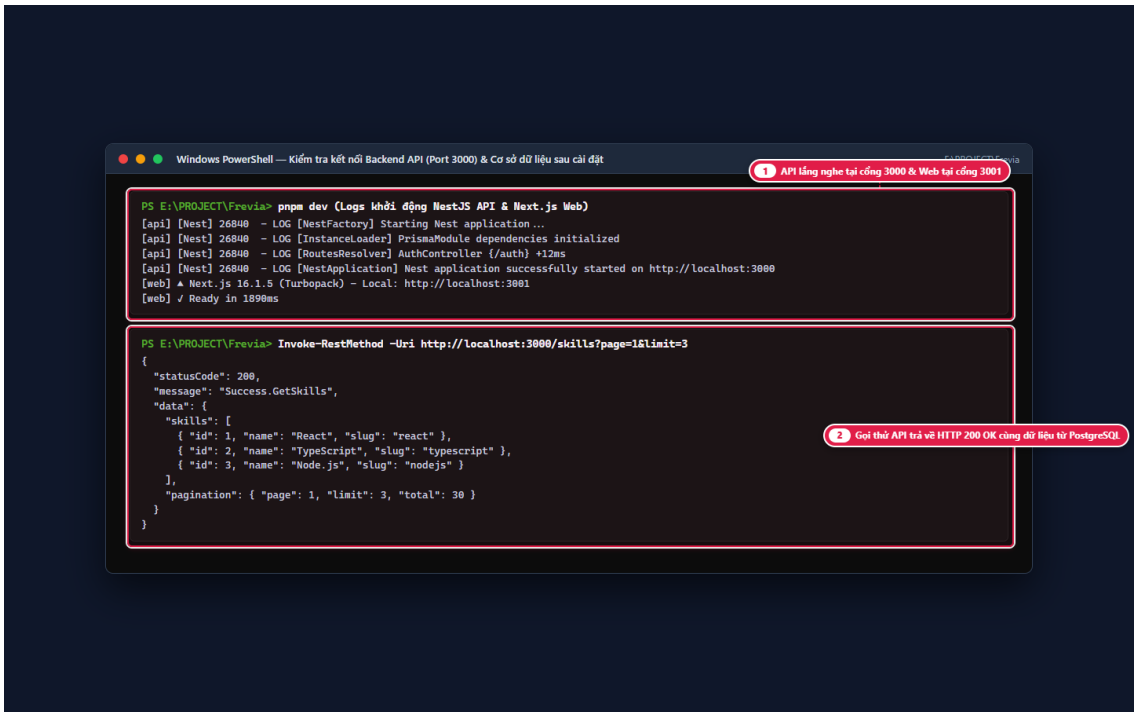


Figure 2.10: Verifying Backend RESTful API endpoints and response payloads via HTTP request

- [1] Query the public skills catalog API:

```
Invoke-RestMethod -Uri "http://localhost:3000/skills" -Method Get
```

  Confirm an **HTTP 200 OK** status returning a structured JSON response array populated with seed skill records.

- [2] Test the Redis cache connection:

```
docker exec -it frevia-redis redis-cli ping
# Expected output: PONG
```

2.7.3 3. Pre-Configured Test Accounts

The database seeder automatically initializes dedicated accounts across all primary system roles to streamline functional evaluation:

Role	Email Address	Default Password	Assigned Access Scope
Administrator	`admin@gmail.com`	`admin123456`	Complete access to `http://localhost:3001/admin` (All 275 permissions).

Role	Email Address	Default Password	Assigned Access Scope
Freelancer	`freelancer@gmail.com`	`freelancer123456`	Profile management, AI CV analysis, proposals, contract milestones, payouts.
Client	`client@gmail.com`	`client123456`	Job posting, candidate review, milestone funding, escrow deposits.
Expert	`expert@gmail.com`	`expert123456`	Expert profile workspace (`/expert/profile`), revision submissions.

3. User Manual

3.1 Overview of the Application & System Actors

Frevia is an enterprise-grade, **AI-Powered Freelance Marketplace and Professional Community Platform** engineered to bridge businesses and elite freelance professionals through automated talent matching, milestone-governed escrow smart contracts, and real-time collaboration tools. Built on a resilient **Role-Based Access Control (RBAC)** architecture, system capabilities and operational permissions are distributed across five primary actor groups:

- **Guest (Unauthenticated Visitor):** Explores public job postings, reviews top-rated freelancer and expert portfolios, browses open community discussions, and executes self-service onboarding or sign-in flows.
- **Freelancer (Independent Talent):** Curates comprehensive multi-faceted profiles, uploads resumes for AI parsing, completes eKYC identity verification, submits bids on posted jobs, negotiates milestone contracts, submits deliverables, and receives payments via Stripe Connect.
- **Client (Employer / Enterprise Buyer):** Manages organization profiles, authors detailed job requisitions, evaluates incoming freelancer proposals, enters into binding milestone contracts, deposits escrow funds, approves deliverables, and submits ratings.
- **Expert (Specialist Consultant):** Operates within an exclusive consultation workspace (`/expert/profile`), submits credentials for administrative revision approvals, and delivers high-tier advisory services.
- **Administrator (System Governor):** Exercises unrestricted governance across the dedicated `/admin` portal: orchestrating dynamic RBAC role-permission assignments, administering multi-profile user directories, maintaining system-wide master catalogs (Skills, Job Categories, Forum Categories), moderating community content, approving eKYC submissions, and mediating milestone dispute arbitrations.

3.2 Module 2: Authentication & Session Management

The Authentication and Session Management module delivers defense-in-depth security infrastructure, featuring: 6-digit email OTP verification, `bcrypt` salted password hashing, stateless `AccessToken` and stateful `RefreshToken` pairs stored in hardened `HttpOnly` cookies, seamless **Google OAuth 2.0** single sign-on, self-service password recovery, and multi-device active session auditing with remote revocation capabilities.

3.2.1 M2.01 – New User Registration & Email OTP Verification (UC-01)

- **Screen Name & Route Path:** User Registration Screen – `/register`.
- **Target Actor:** Guest (New visitor without an existing account).
- **Preconditions:** The user is not currently authenticated and possesses an accessible email address not previously registered on Frevia.

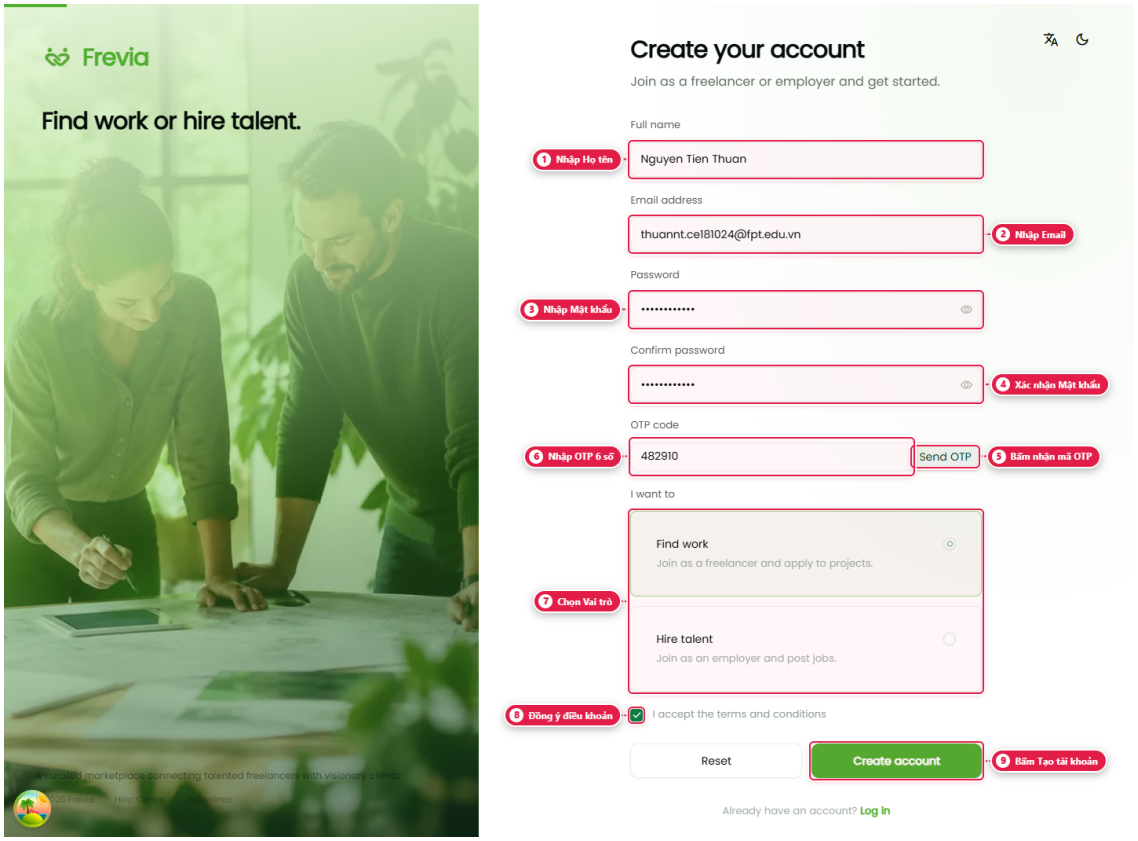


Figure 3.1: User Registration and Email OTP Verification interface at `/register`

Step-by-Step User Instructions:

1. **[1] Enter Full Name (*Full name*):** Type the legal or professional full name into the *Full name* field (maximum 100 characters, e.g., `Nguyen Tien Thuan`).
2. **[2] Enter Email Address (*Email address*):** Type a valid personal or business email address into the *Email address* field.

3. **[3] Enter Password (*Password*):** Type a strong password (minimum 6 characters). Users may click the eye icon on the right to toggle password visibility.
4. **[4] Confirm Password (*Confirm password*):** Retype the exact password entered in step 3 to prevent typographical errors.
5. **[5] Request Verification OTP (*Send OTP*):** Click the blue **Send OTP** button adjacent to the OTP field. The server generates a cryptographic 6-digit code, caches it in Redis with a 5-minute time-to-live (TTL), dispatches it via Gmail SMTP, and activates a 60-second cooldown timer.
6. **[6] Enter Verification OTP (*OTP code*):** Inspect the email inbox, copy the 6-digit verification code, and enter it into the *OTP code* field.
7. **[7] Select Account Intention (*I want to*):** Choose an initial operational role:
    - **Find work (Freelancer):** Register as a service provider to build a portfolio and submit job proposals.
    - **Hire talent (Client):** Register as an enterprise or individual client to publish jobs and contract talent.
8. **[8] Accept Terms of Service (*Accept terms*):** Check the *I accept the terms and conditions* checkbox to fulfill legal compliance requirements and enable form submission.
9. **[9] Submit Registration Form (*Create account*):** Click the blue **Create account** button to submit the validated payload (or click *Reset* to clear all fields).

Expected Output & Exception Handling:

- **Success Scenario:** The server provisions a new `User` record, links the chosen role (`FREELANCER` or `CLIENT`), flags `isEmailVerified = true`, presents a green success toast notification, and navigates to `/login`.
- **Exception Scenarios:**
    - If the email is already registered, an error toast displays: *"Email already exists in the system"*.
    - If the OTP is expired or incorrect, an error alert displays: *"Invalid or expired verification code"*.
    - If passwords do not match, client-side Zod validation flags the confirmation field immediately.

3.2.2 M2.02 – User Authentication via Email & Password (UC-02)

- **Screen Name & Route Path:** User Sign In Screen – `/login`.
- **Target Actor:** Registered User (Guest).
- **Preconditions:** The account has been previously registered and is currently in good standing (`isBanned = false`).

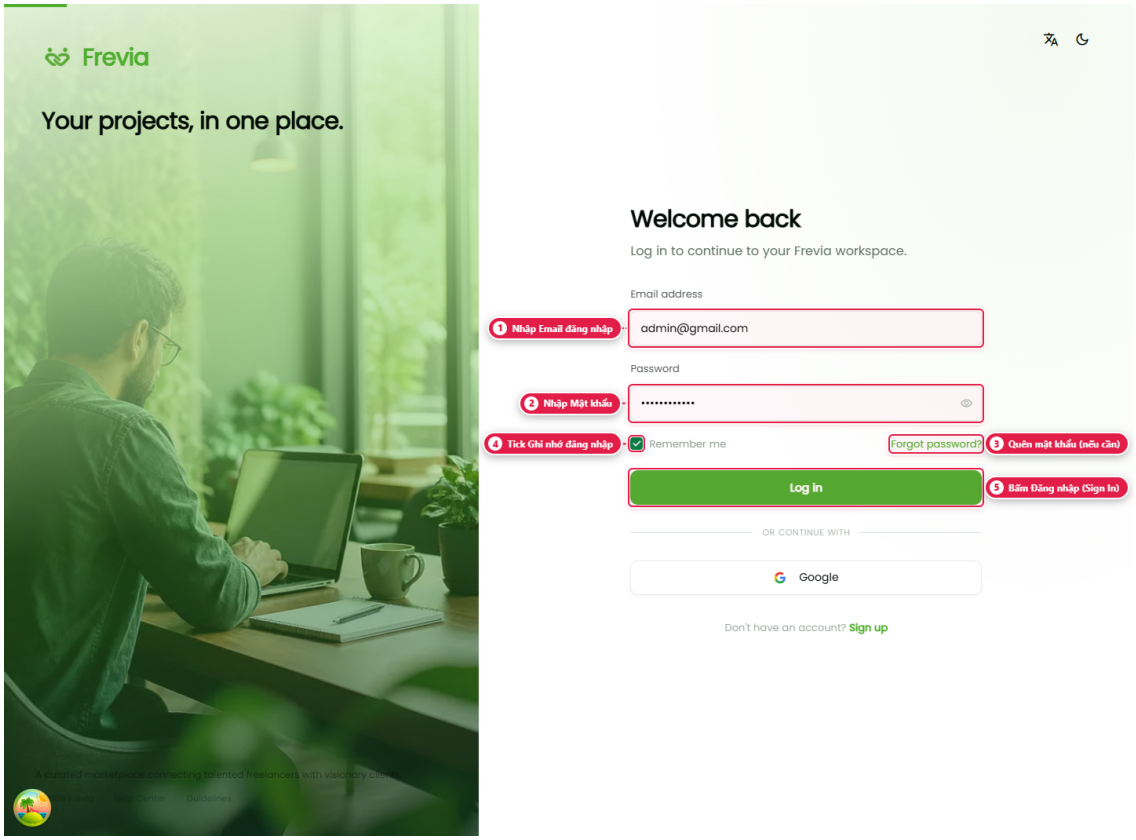


Figure 3.2: User Login authentication screen at `/login`

Step-by-Step User Instructions:

1. **[1] Enter Registered Email (*Email*):** Input the account email into the *Email* field (e.g., `admin@gmail.com` or `freelancer@gmail.com`).
2. **[2] Enter Password (*Password*):** Input the account password into the *Password* field.
3. **[3] Toggle Persistent Session (*Remember me*):** Check the *Remember me* checkbox to extend session persistence across browser restarts.
4. **[4] Password Recovery Link (*Forgot password?*):** If credentials have been forgotten, click the *Forgot password?* link to initiate the recovery flow.
5. **[5] Click Sign In (*Sign in*):** Click the blue **Sign in** button to authenticate.

Expected Output & Exception Handling:

- **Success Scenario:** The backend verifies the `bcrypt` hash, creates an active `Session` entity recording client IP, User-Agent, and device metadata, sets secure `HttpOnly` cookies, and redirects the user:
  - Accounts holding the `ADMIN` role are redirected to the administrator console at `/admin`.
  - Freelancers and Clients are redirected to their tailored home dashboard.
- **Exception Scenarios:**
  - If credentials do not match, the application presents an error alert: *"Invalid email or password"*.
  - If the account is suspended, an alert states: *"Your account has been suspended. Please contact support"*.

3.2.3 M2.03 – Single Sign-On via Google OAuth 2.0 (UC-03)

- **Screen Name & Route Path:** Social Sign-In Interface – `/login`.
- **Target Actor:** Guest.
- **Preconditions:** The user possesses an active Google account.

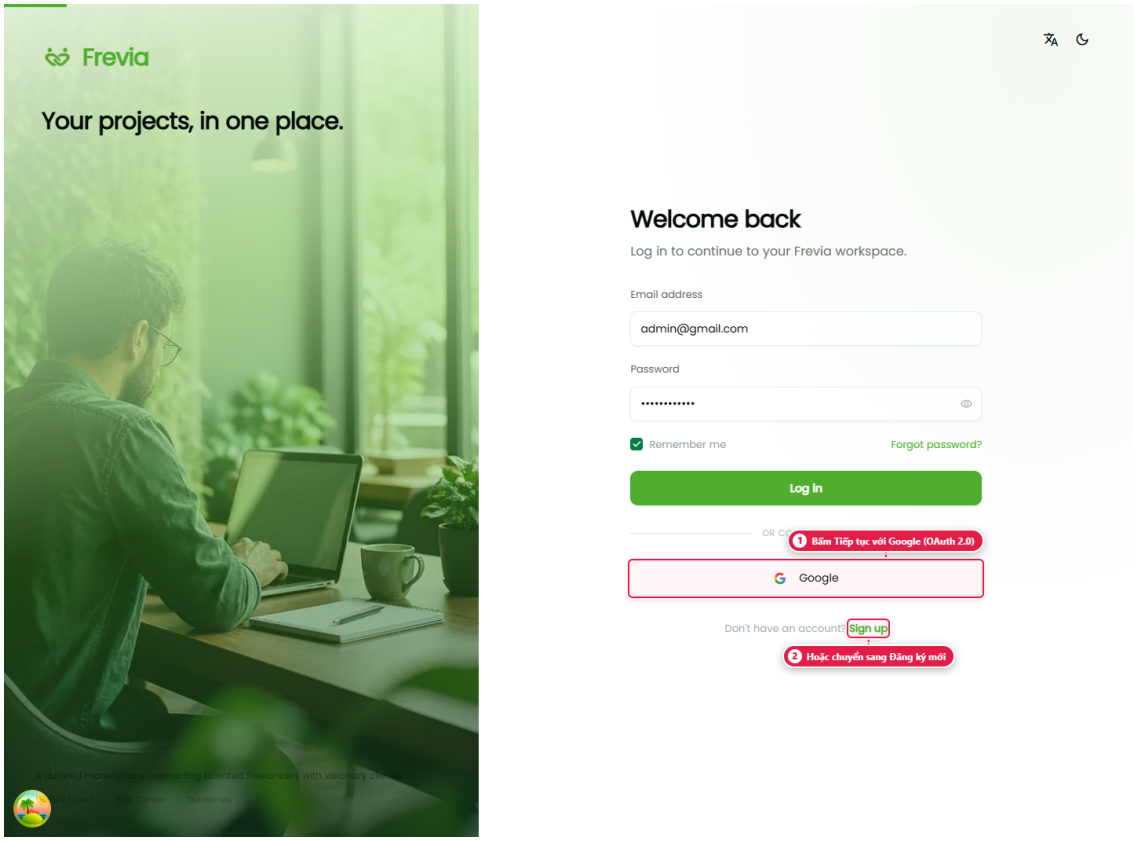


Figure 3.3: Google OAuth 2.0 Single Sign-On button and registration link on the login screen

Step-by-Step User Instructions:

1. **[1] Click Google Sign-In Button (*Continue with Google*):** Click the **Google** button beneath the divider to initiate federated authentication via the standard Google OAuth 2.0 consent flow.
2. **[2] Registration Navigation Link (*Sign up*):** Alternatively, if the user does not possess an account, clicking *Sign up* immediately routes to the registration view at `/register`.

Expected Output & Exception Handling:

- **Success Scenario:** Google validates identity and redirects the browser back to `/api/auth/google/callbac` The backend links or provisions the user profile, sets authentication cookies, and signs the user in instantly.
- **Exception Scenarios:** If the user cancels the Google permission prompt, the callback redirects back to `/login` with a cancellation notification.

3.2.4 M2.04 – Password Recovery & Reset via Email OTP (UC-04)

- **Screen Name & Route Path:** Password Recovery Screen – `/forgot-password`.
- **Target Actor:** Registered User who has forgotten their account password.
- **Preconditions:** The user knows the registered email address associated with their Frevia account.

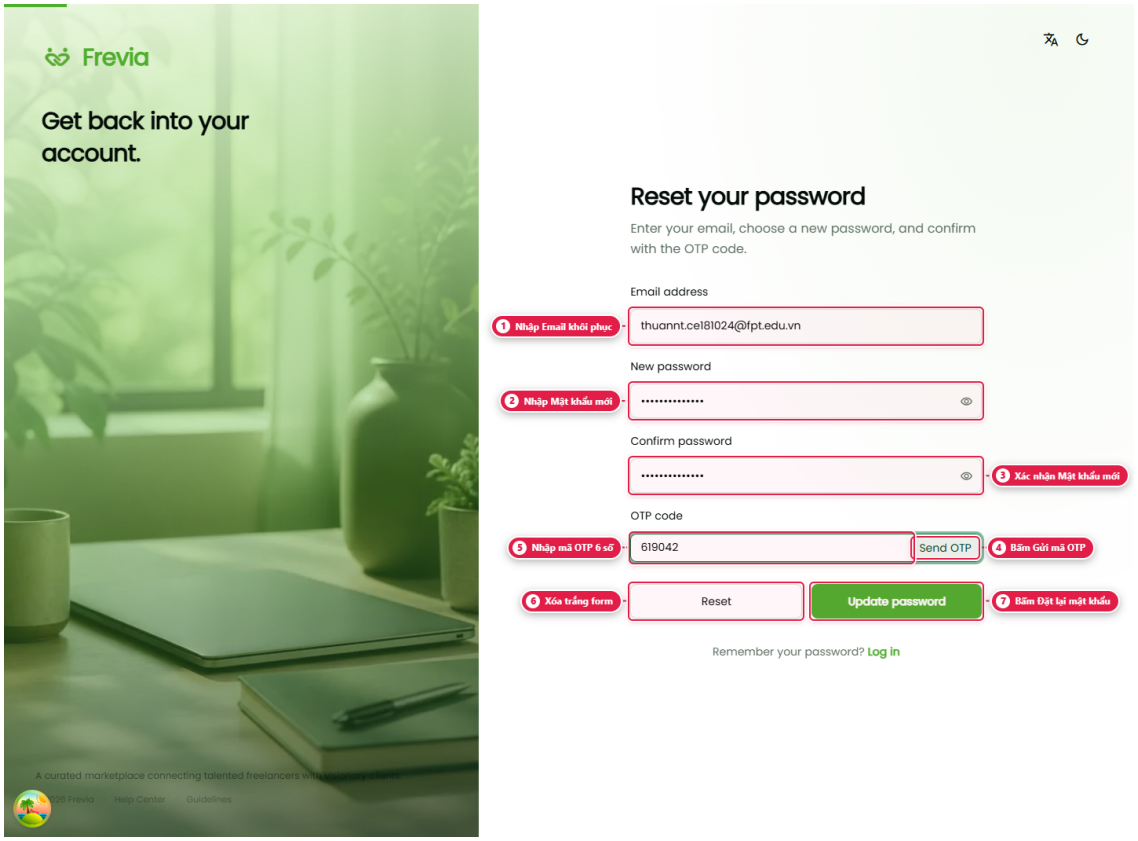


Figure 3.4: Password recovery and OTP reset interface at `/forgot-password`

Step-by-Step User Instructions:

1. **[1] Enter Account Email (*Email address*):** Type the registered email into the *Email address* field.
2. **[2] Request Reset OTP (*Send OTP*):** Click the blue **Send OTP** button to dispatch a reset code to the mailbox.
3. **[3] Enter Verification OTP (*OTP code*):** Retrieve the 6-digit code from the received email and input it into the *OTP code* field.
4. **[4] Enter New Password (*New password*):** Type the replacement password (minimum 6 characters).
5. **[5] Confirm New Password (*Confirm new password*):** Retype the new password to ensure consistency.
6. **[6] Submit Password Reset (*Reset password*):** Click the blue **Reset password** button to commit changes.

7. **[7] Return to Sign In Link (*Back to sign in*):** Click **Back to sign in** to return to `/login` without resetting.

Expected Output & Exception Handling:

- **Success Scenario:** The server validates the OTP against Redis, re-hashes the new password with bcrypt, invalidates all existing active sessions for security, displays a confirmation toast, and redirects to `/login`.
- **Exception Scenarios:** If the OTP is invalid or expired, the submission is rejected with an explicit error alert.

3.2.5 M2.05 – Active Login Session Monitoring & Device Audit (UC-05.01)

- **Screen Name & Route Path:** Session Management Console – `/sessions`.
- **Target Actor:** Authenticated User (Freelancer, Client, Expert, Admin).
- **Preconditions:** User is logged into the application with at least one active session record in the database.

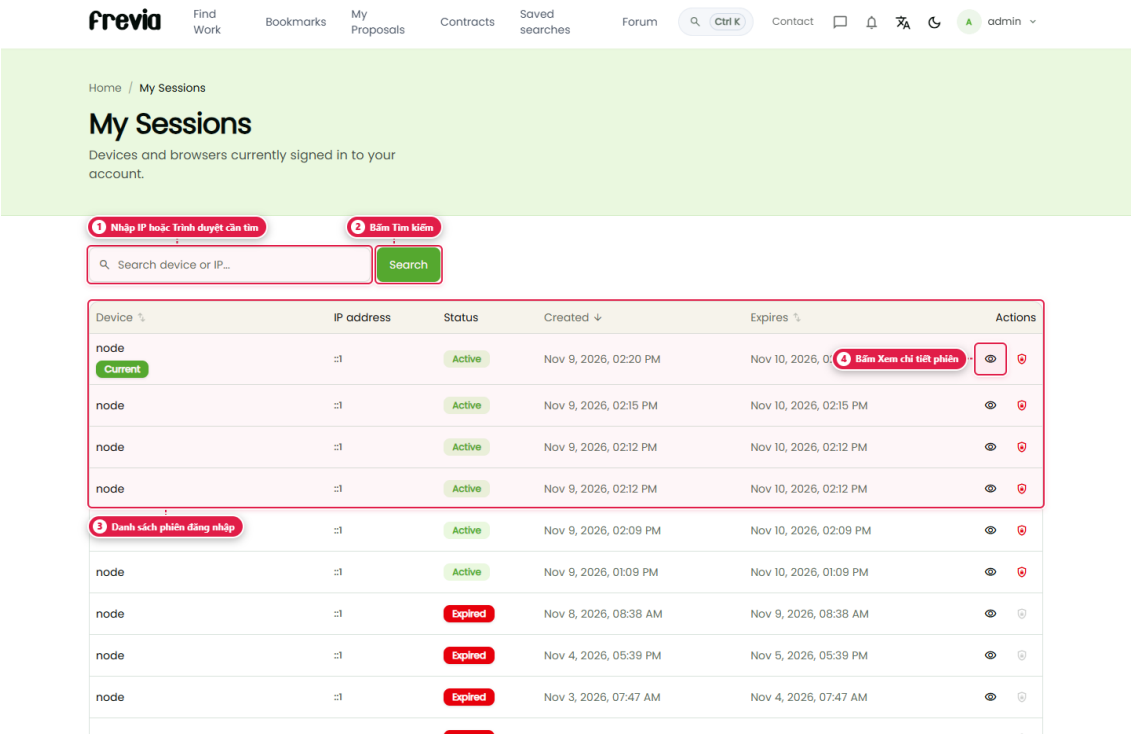


Figure 3.5: Active login sessions audit table with search and filtering at `/sessions`

Step-by-Step User Instructions:

1. Navigate to `http://localhost:3001/sessions` from the user settings menu.
2. **[1] Filter Sessions by Device / IP (*Search input*):** Type a browser name, operating system, or IP address into the search box to dynamically filter listed records.
3. **[2] Inspect Active Session Records (*Sessions table*):** Review session attributes across columns: *Device / Browser* (detected User-Agent), *IP Address*, *Location*, *Created* timestamp, and *Status* (the active browser instance displays a blue **Current Session** badge).
4. **[3] Revoke Remote Session Button (*Trash icon*):** Click the red trash icon on any remote session row to disconnect that specific device.

Expected Output & Exception Handling:

- **Success Scenario:** The user gains full visibility over all authorized devices, identifying potential unauthorized account access.

3.2.6 M2.06 – Remote Session Revocation & Security Enforcement (UC-05.02)

- **Screen Name & Route Path:** Remote Session Termination Confirmation Dialog – `/sessions`.
- **Target Actor:** Authenticated User.
- **Preconditions:** At least one non-current active session exists in the sessions table.

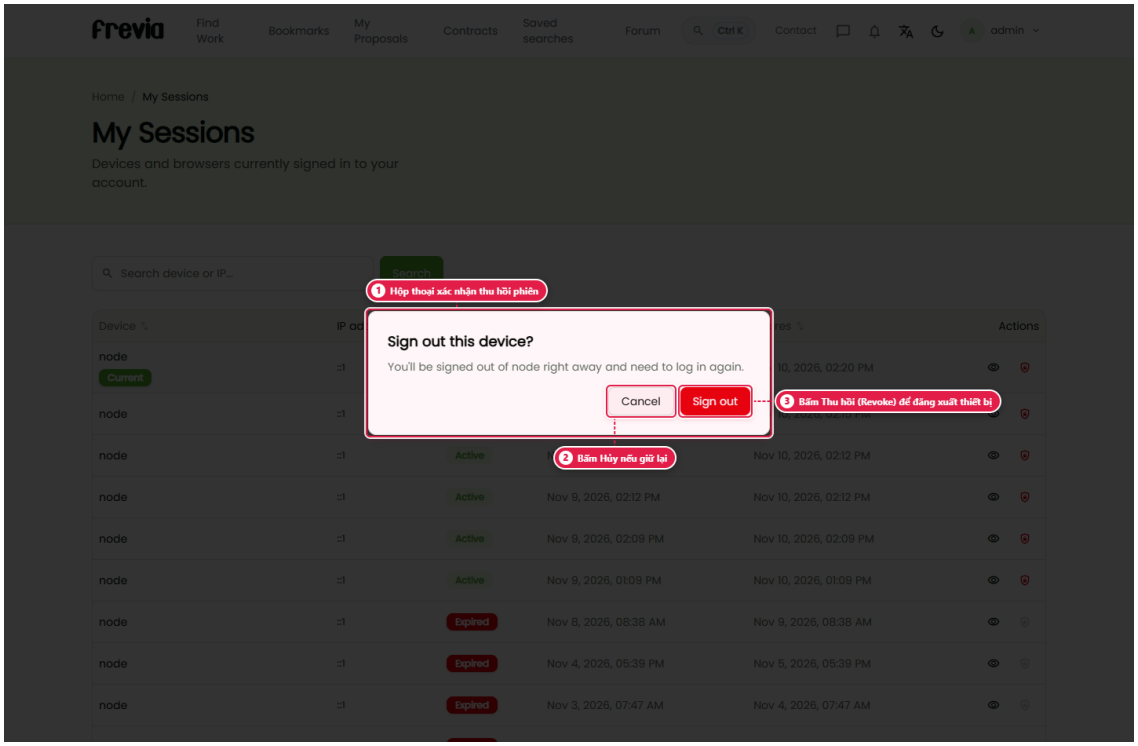


Figure 3.6: Confirmation modal dialog for revoking a remote session at `/sessions`

Step-by-Step User Instructions:

1. **[1] Click Revoke Session Button:** In the session table, locate the target remote device row and click its red trash icon.
2. **[2] Read Revocation Warning (*Dialog title & description*):** An `AlertDialog` appears reading: *"Revoke Session – Are you sure you want to revoke this session? The device will be logged out immediately."*
3. **[3] Confirm Revocation Action (*Revoke button*):** Click the red **Revoke** button to confirm immediate termination (or click *Cancel* to dismiss the dialog without taking action).

Expected Output & Exception Handling:

- **Success Scenario:** The backend invalidates the corresponding `RefreshToken` in PostgreSQL and purges the token from Redis. The targeted device will have its session rejected upon its subsequent API request and will be redirected to `/login`. A green toast confirms: *"Session revoked successfully"*.

3.2.7 M2.07 – Secure User Sign-Out & Session Invalidation (UC-05.03)

- **Screen Name & Route Path:** Sidebar / Header Navigation across all portals.
- **Target Actor:** Authenticated User.
- **Preconditions:** User is currently logged into the system.

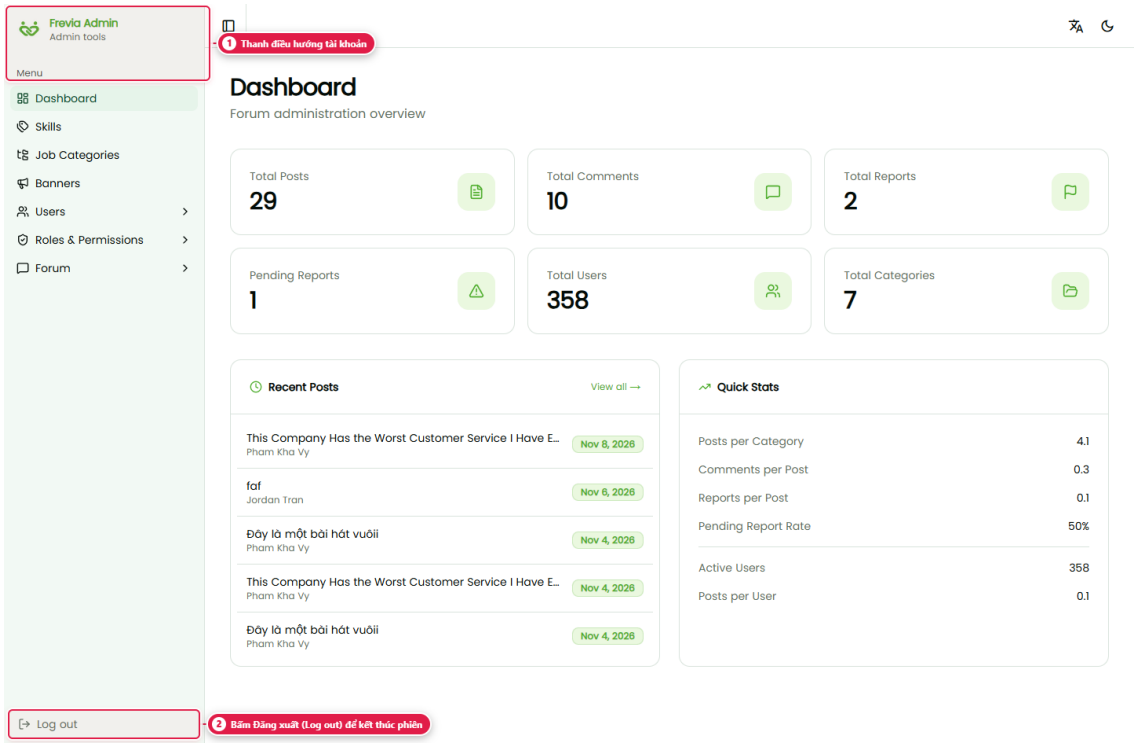


Figure 3.7: User identity card and sign-out button located in the application navigation bar

Step-by-Step User Instructions:

1. **[1] Inspect User Profile Summary:** In the lower left section of the sidebar navigation, observe the authenticated identity card showing Avatar, Full Name (`admin`), Email (`admin@gmail.com`), and role badge.
2. **[2] Click Sign Out Button (*Logout icon*):** Click the red exit icon located on the right side of the identity card.

Expected Output & Exception Handling:

- **Success Scenario:** The application dispatches an HTTP POST request to `/api/auth/logout`, expires client `HttpOnly` cookies, purges cached application state from TanStack Query, and redirects the browser cleanly to `/login`.

3.3 Module 14: Administrator System, RBAC & User Management

The Administrator System, RBAC, and User Management module empowers system governors with a centralized management console (*Admin Portal*). From this single pane of glass, administrators can monitor cross-platform operational metrics, configure fine-grained **Role-Based Access Control (RBAC)** down to specific API endpoints (*HTTP Method + Path*), and oversee user account lifecycles along with their specialized multi-profile identities.

3.3.1 M14.01 – Administrator Central Operations Dashboard (`/admin`)

- **Screen Name & Route Path:** Administrator Dashboard – `/admin`.
- **Target Actor:** Administrator.
- **Preconditions:** Authenticated user holds the `ADMIN` role in good standing.

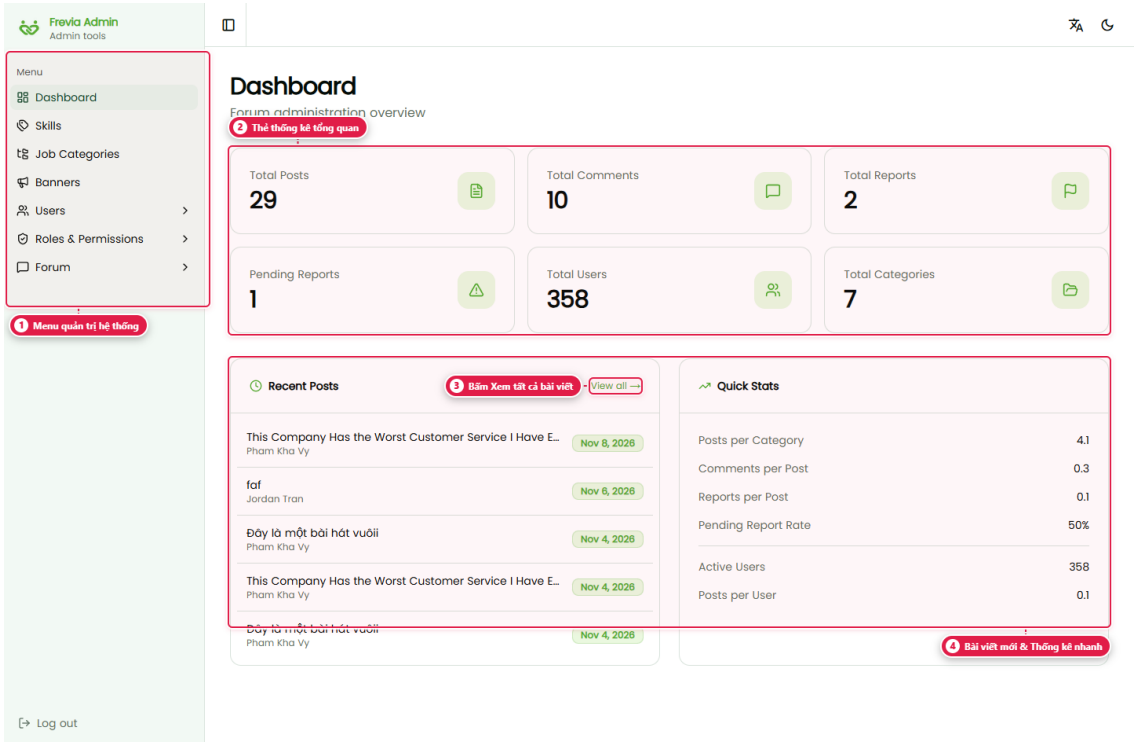


Figure 3.8: Central Administrator Dashboard and operational KPI metrics at `/admin`

Step-by-Step User Instructions:

1. **[1] Navigate via Admin Sidebar (*Navigation menu*):** Utilize the left sidebar to navigate across management subsystems: *Dashboard*, *Skills*, *Job Categories*, *Banners*, the *Users* group (*User Management*, *Identity Verification*, *Profile Reviews*), the *Roles & Permissions* group (*Roles*, *Permissions*, *Assign Role*), and the *Forum* group.
2. **[2] Review Platform Operational KPIs (*Overview stat cards*):** Monitor 6 real-time operational indicators: *Total Posts*, *Total Comments*, *Total Reports*, *Pending Reports*, *Total Users*, and *Total Categories*.

3. **[3] Navigate to Posts Catalog (*View all* →):** In the *Recent Posts* panel, inspect recent publications, authors, and timestamps; click **View all** → to access the comprehensive moderation table at `/admin/posts`.
4. **[4] Inspect Analytics Breakdown (*Quick stats*):** Examine platform ratios on the right panel, including *Posts per Category*, *Comments per Post*, *Pending Report Rate*, and current *Active Users*.

Expected Output & Exception Handling:

- **Success Scenario:** All analytics synchronize dynamically with the underlying database.
- **Exception Scenarios:** If an unprivileged user navigates to `/admin`, the authorization guard intercepts the request and redirects to `/access-denied?requiredRole=admin`.

3.3.2 M14.02 – Administrator Profile, Identity & System Privileges (`/admin/users/1`)

- **Screen Name & Route Path:** Administrator Profile & Privileges View – `/admin/users/1`.
- **Target Actor:** Administrator.
- **Preconditions:** Administrator is signed in and authorized.

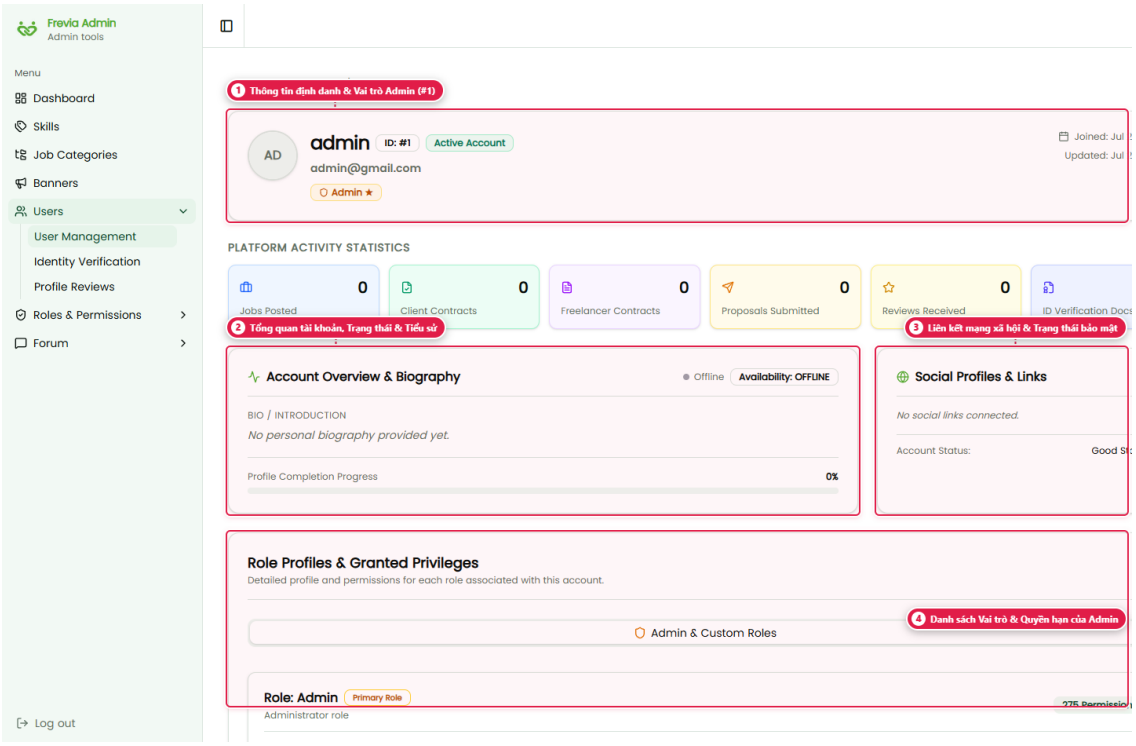


Figure 3.9: Administrator profile identity, standing, and granted privileges at `/admin/users/1`

Step-by-Step User Instructions:

1. **[1] Review Core Identity Information (#1):** Inspect the primary identity card displaying Avatar, Display Name (`admin`), User ID (`ID: #1`), account standing badge (**Active Account**), Email (`admin@gmail.com`), and the **Admin** ⋆ role badge.
2. **[2] Inspect Account Standing & Bio (*Overview & Bio*):** Review online connectivity status, biographical summary, and profile completeness indicator.
3. **[3] Review Social Links & Security Standing (*Social Profiles*):** Verify linked social identity bindings and account trust scores.
4. **[4] Audit Assigned System Privileges (*Role Profiles & Privileges*):** Click the **Admin & Custom Roles** tab to review the complete list of all 275 assigned API permissions. Administrator details can be updated via the *Edit user* dialog on the user management page (`/admin/users`).

Expected Output & Exception Handling:

- **Success Scenario:** Detailed administrative authority and system credentials are transparently presented for auditability.

3.3.3 M14.03 – System Roles & Custom Roles Management (UC-06.01 to UC-06.05)

- **Screen Name & Route Path:** Roles Management Console – `/admin/roles`.
- **Target Actor:** Administrator.
- **Preconditions:** Access to `ADMIN_ROLES` module permissions.

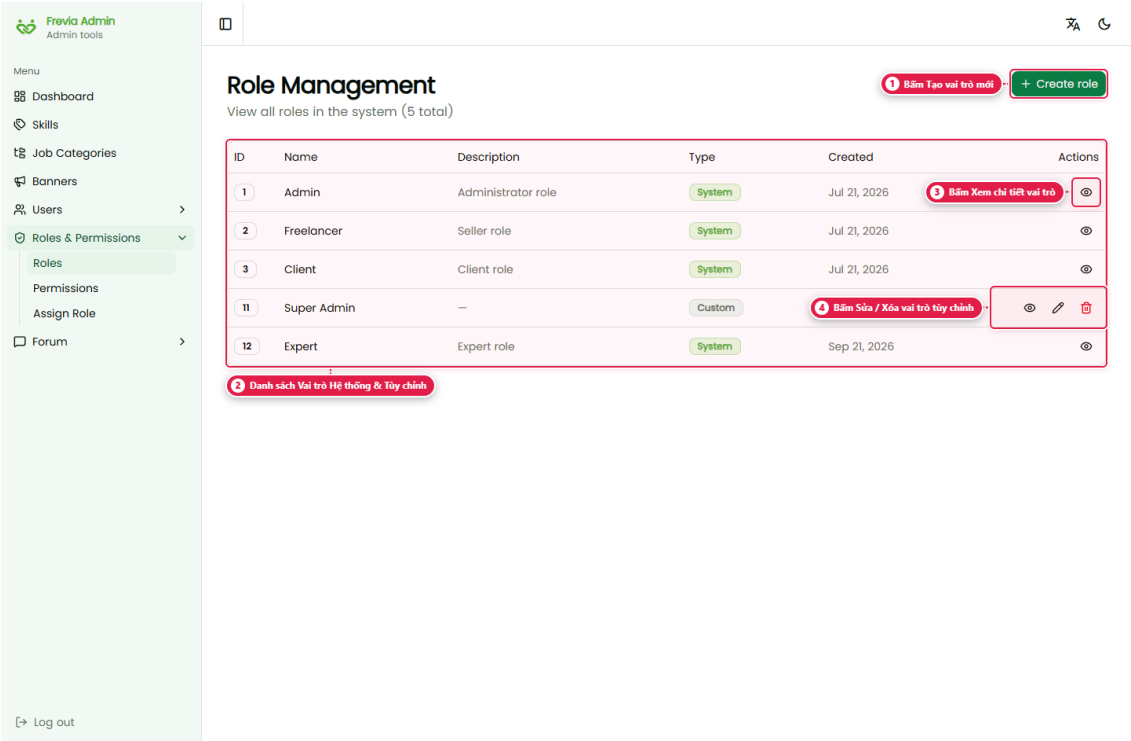


Figure 3.10: System and Custom Roles management console at `/admin/roles`

Step-by-Step User Instructions:

1. In the Admin sidebar, navigate to **Roles & Permissions** → **Roles**.
2. **[1] Search Roles (*Search input*):** Filter records by role name (e.g., `ADMIN`, `FREELANCER`, `CLIENT`, `EXPERT`, `MODERATOR`).
3. **[2] Create New Role (+ *Create Role*):** Click **+ Create Role** to launch a dialog where custom roles (e.g., *Moderator*, *Auditor*) can be defined with a Name and Description.
4. **[3] Inspect Roles Table (*Roles Table*):** View listed roles with their *ID*, *Name*, *Description*, *Type* (distinguishing unalterable core **System Roles** from user-created **Custom Roles**), and creation timestamps.
5. **[4] Perform Role Actions (*Actions column*):**
   - Click the shield key icon (**Shield icon**) to open the permission assignment matrix for that role.
   - Click the pencil icon (**Pencil icon**) to update custom role metadata.
   - Click the red trash icon (**Trash icon**) to delete custom roles. Core system roles are locked to prevent accidental deletion.

Expected Output & Exception Handling:

- **Success Scenario:** New roles are immediately available across the user management and permission assignment interfaces.
- **Exception Scenarios:** The backend enforces a uniqueness constraint on role names and rejects attempts to delete default system roles.

3.3.4 M14.04 – Dynamic Role-Permission Assignment Matrix (UC-06.06)

- **Screen Name & Route Path:** Role Permission Assignment Matrix – `/admin/assign-role`.
- **Target Actor:** Administrator.
- **Preconditions:** Administrator has permissions to assign RBAC rights.

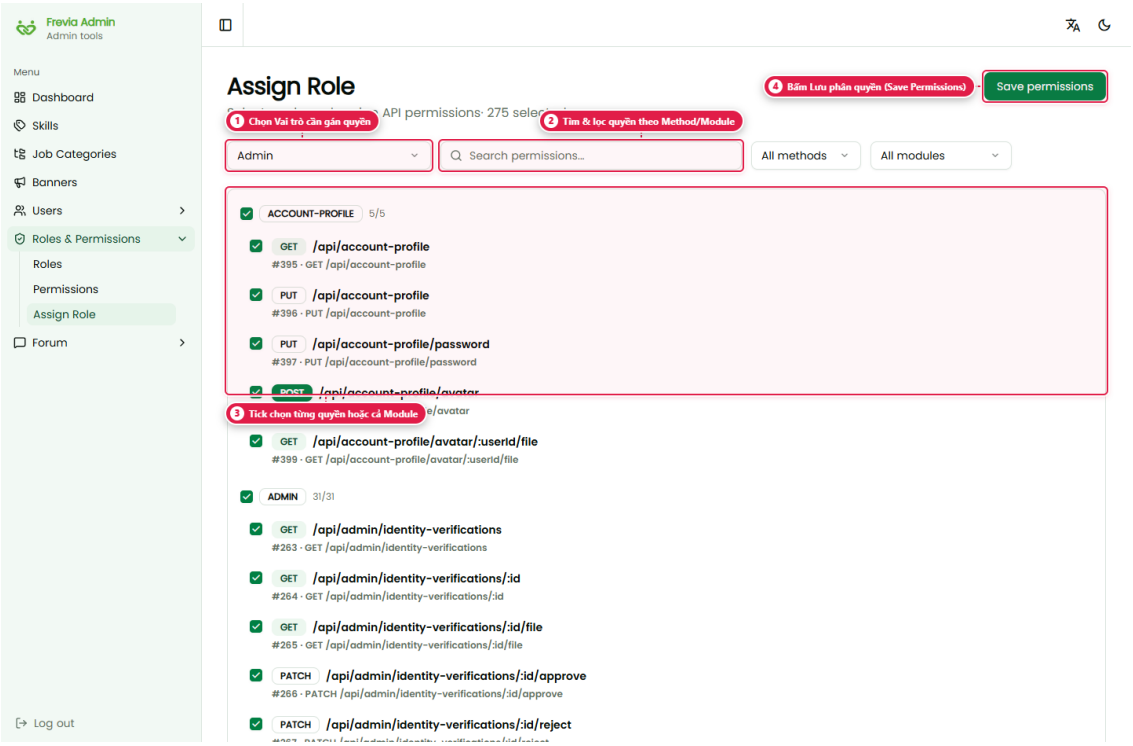


Figure 3.11: Dynamic RBAC role-permission matrix configuration at `/admin/assign-role`

Step-by-Step User Instructions:

1. In the Admin sidebar, navigate to **Roles & Permissions** → **Assign Role**.
2. **[1] Select Target Role (*Select Role dropdown*):** Choose the role to configure (e.g., `ADMIN`, `FREELANCER`, `CLIENT`, `EXPERT`).
3. **[2] Persist Matrix Changes (*Save Changes*):** After toggling permissions, click the green **Save Changes** button to commit updates to the database.
4. **[3] Toggle Permissions by Module (*Permission Matrix*):** Scroll through permission groups organized by domain modules (*AUTH*, *USERS*, *ROLES*, *SKILLS*, *JOBS*, *PROPOSALS*, *CONTRACTS*, *PAYMENTS*, *REVIEWS*). Toggle switches to grant or revoke specific API endpoints.
5. **[4] Batch Module Toggles (*Select All / Deselect All*):** Click module-level headers to toggle all contained permissions in a single action.

Expected Output & Exception Handling:

- **Success Scenario:** The server updates the `RolePermission` junction table. NestJS guards evaluate these updated permissions on subsequent requests without requiring server reboots.

3.3.5 M14.05 – System API Permission Catalog & Multi-Criteria Filtering (UC-15.01 to UC-15.04)

- **Screen Name & Route Path:** Permissions Catalog – `/admin/permissions`.
- **Target Actor:** Administrator.
- **Preconditions:** Access to `ADMIN_PERMISSIONS` module.

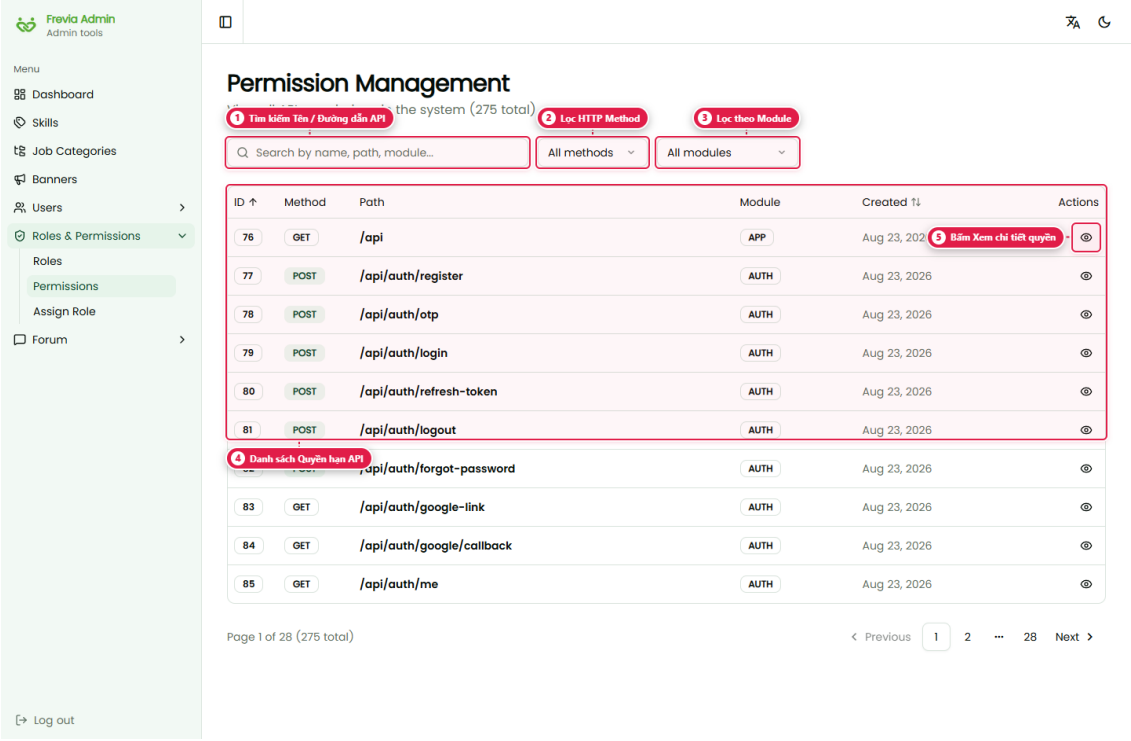


Figure 3.12: System API Permission catalog with method and module filters at `/admin/permissions`

Step-by-Step User Instructions:

1. In the Admin sidebar, click **Roles & Permissions** → **Permissions**.
2. **[1] Search Permissions by Name or Path (*Search box*):** Input keywords matching permission names (e.g., `CREATE_JOB`), paths (e.g., `/jobs`), or descriptions.
3. **[2] Filter by HTTP Method (*Method filter*):** Select an HTTP verb from the dropdown (*ALL*, *GET*, *POST*, *PUT*, *PATCH*, *DELETE*) to isolate specific operations.
4. **[3] Filter by Domain Module (*Module filter*):** Select a functional module (e.g., *AUTH*, *USERS*, *JOBS*, *CONTRACTS*, *PAYMENT*) to view its endpoints.
5. **[4] Inspect Permissions Table (*Permissions table*):** View permission records with *ID*, *Name*, color-coded *Method* badges (Blue for GET, Green for POST, Amber for PUT, Red for DELETE), *Path*, and *Module*.

Expected Output & Exception Handling:

- **Success Scenario:** Instantaneous client-side and server-side filtering across the entire catalog of 275 system endpoints.

3.3.6 M14.06 – User Directory Administration & Direct Account Provisioning (UC-07.01 to UC-07.03)

- **Screen Name & Route Path:** User Management Console – `/admin/users`.
- **Target Actor:** Administrator.
- **Preconditions:** Access to `ADMIN_USERS` module.

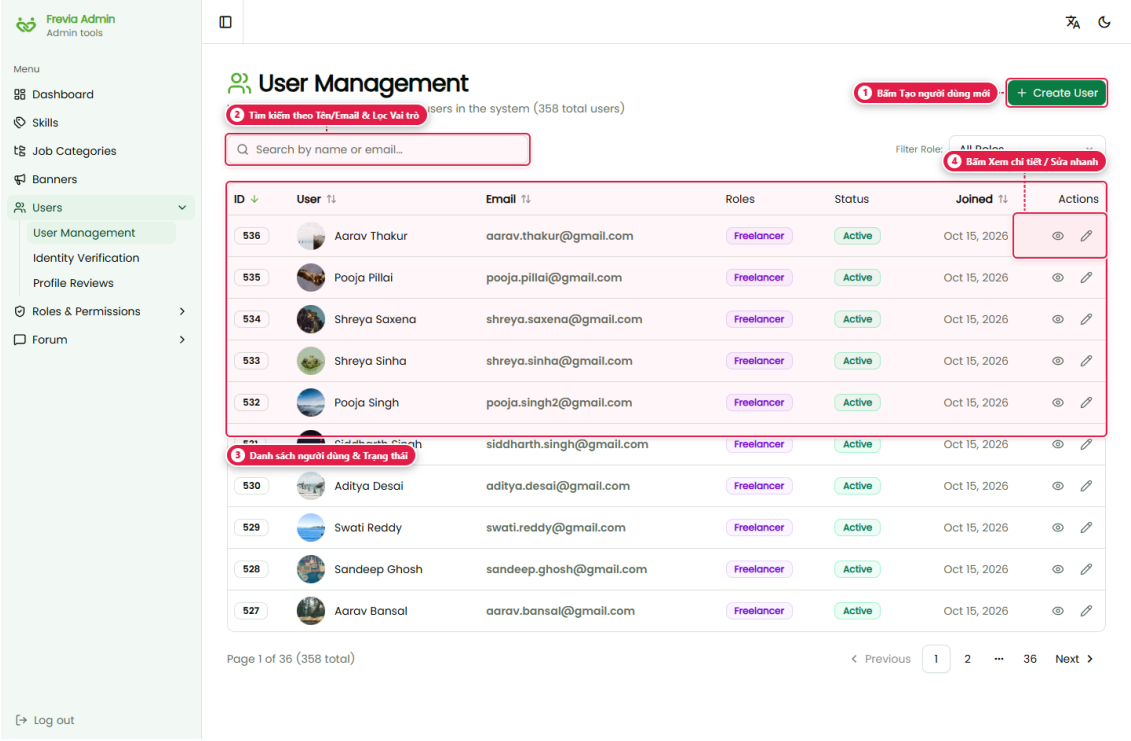


Figure 3.13: User directory table and account provisioning interface at `/admin/users`

Step-by-Step User Instructions:

1. In the Admin sidebar, click **Users** → **User Management**.
2. **[1] Search Users by Name or Email (*Search input*):** Type a query to search across user records in real time.
3. **[2] Filter Accounts by Standing (*Status filter*):** Select *All*, *Active*, or *Banned* from the status dropdown.
4. **[3] Direct Account Provisioning (+ *Create User*):** Click **+ Create User** to open the modal dialog, input Full Name, Email, Password, and assign initial roles.
5. **[4] Inspect User Table & Row Actions (*Users Table & Actions*):** Review user attributes (*ID*, *Name*, *Email*, *Status*, *Roles*, *Created*). On each row:
   - Click the eye icon (**Eye icon**) to open the user's multi-profile details view (`/admin/users/[id]`).
   - Click the pencil icon (**Pencil icon**) to open the *Edit user* modal dialog.

Expected Output & Exception Handling:

- **Success Scenario:** Administrators maintain full directory oversight and can provision verified staff accounts directly.

3.3.7 M14.07 – User Account Status Management & Suspension/Reactivation (UC-07.04 to UC-07.05)

- **Screen Name & Route Path:** Edit User Dialog – `/admin/users`.
- **Target Actor:** Administrator.
- **Preconditions:** Target user account exists in the system.

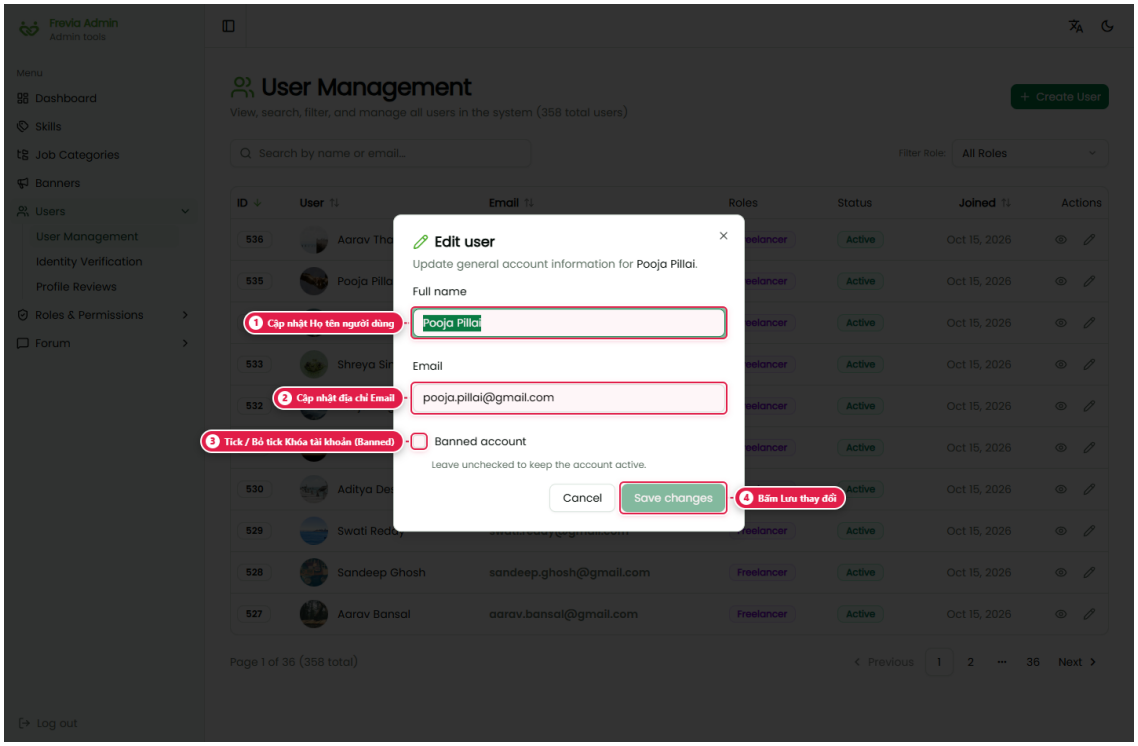


Figure 3.14: Edit User modal dialog featuring the Banned account toggle at `/admin/users`

Step-by-Step User Instructions:

1. **[1] Open Edit User Dialog:** Click the pencil icon on any user row in `/admin/users`. The modal header displays *"Edit User – Update user profile and manage account status"*.
2. **[2] Edit Full Name (*Full Name field*):** Modify the user's name if required.
3. **[3] Edit Email Address (*Email field*):** Update the primary email address if necessary.
4. **[4] Toggle Account Suspension (*Banned account checkbox*):**
   - **Suspend Account:** Check *"Banned account – Check to ban this user from accessing the system"*.
   - **Reactivate Account:** Uncheck the box to restore the user account to **Active** status.
   - Click **Save changes** to apply the update.

Expected Output & Exception Handling:

- **Success Scenario:** Setting `isBanned = true` immediately invalidates all active sessions for that user. Subsequent sign-in attempts by that account will be blocked with an account suspension notification.

3.3.8 M14.08 – Comprehensive User Multi-Profile Management (UC-07.06 to UC-07.07)

- **Screen Name & Route Path:** Multi-Profile Inspection View – `/admin/users/[id]`.
- **Target Actor:** Administrator.
- **Preconditions:** User holds multi-role profile records (e.g., User #2 holding Client and Freelancer identities).

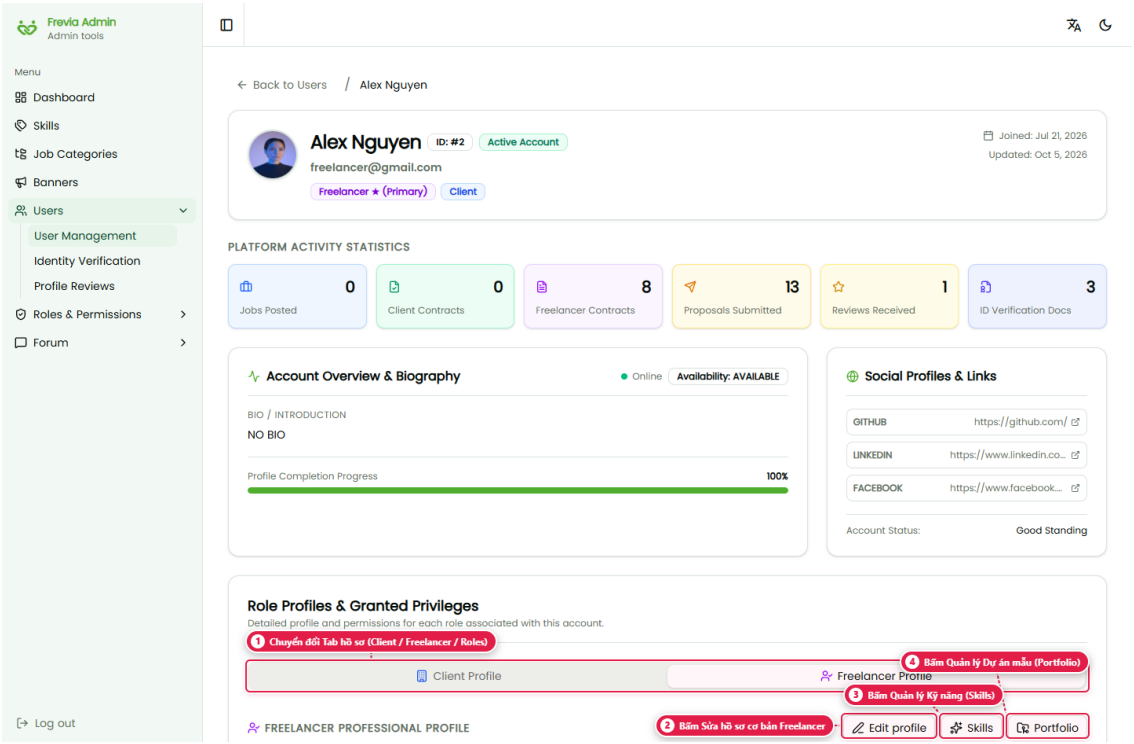


Figure 3.15: Admin inspection view of dual Client and Freelancer profiles at `/admin/users/2`

Step-by-Step User Instructions:

1. Navigate to `/admin/users/2` (or click the eye icon for that user in the directory).
2. **[1] Inspect Multi-Role Identity Badges:** Review the user header displaying Avatar, Display Name, Email, and active role badges (**Client**, **Freelancer**).
3. **[2] Switch Between Profile Tabs (*Navigation tabs*):** Select from the available domain tabs: **Client Profile**, **Freelancer Profile**, **Skills & Experience**, and **Portfolio**.
4. **[3] Inspect Client Profile Metrics (*Client profile overview*):** Under *Client Profile*, examine Company Name, Website, Payment Verification Status, Total Posted Jobs, and Total Spend.
5. **[4] Inspect Freelancer Profile Metrics (*Freelancer profile overview*):** Under *Freelancer Profile*, examine Professional Title, Hourly Rate, Completed Jobs, Total Earnings, and Bio.

Expected Output & Exception Handling:

- **Success Scenario:** The administrator can seamlessly inspect and audit both sides of a dual-role user account without having to log in as that user.

3.4 Module 15: Administrator Master Data Catalog Management

The Administrator Master Data Catalog Management module empowers system administrators to maintain foundational classification dictionaries across **Frevia**, including: Technical Skills catalogs (**Skills**), Job & Industry categories (**Job Categories**), and Community Forum topic boards (**Forum Categories**). To maintain data integrity across active job postings, freelancer portfolios, and historical community discussions, all catalog items utilize a non-destructive **Soft Delete and Restore** lifecycle architecture.

3.4.1 M15.01 – Skills Catalog Management with Soft Delete & Restore (UC-11.01 to UC-11.06)

- **Screen Name & Route Path:** Skills Management Console – `/admin/skills`.
- **Target Actor:** Administrator.
- **Preconditions:** Administrator has permissions to access the `ADMIN_SKILLS` module.

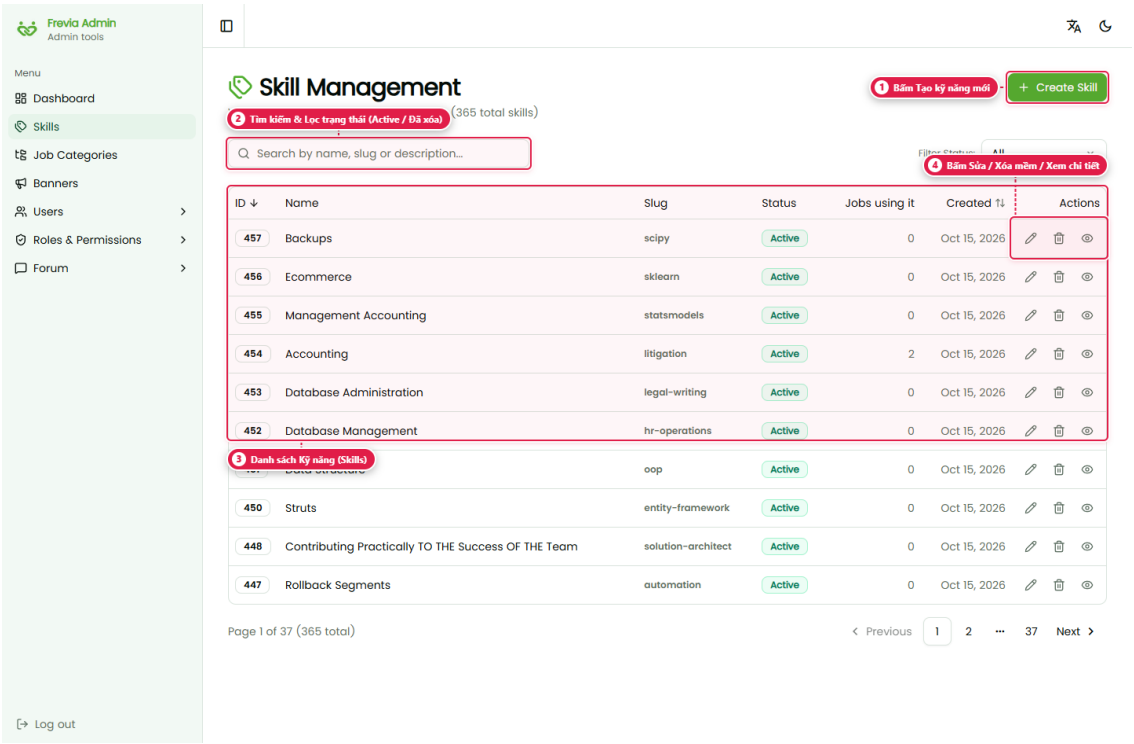


Figure 3.16: Skills catalog management console with search and soft delete at `/admin/skills`

Step-by-Step User Instructions:

1. In the Admin sidebar, click **Skills**.
2. **[1] Create New Skill (+ *Create Skill*):** Click the blue **+ Create Skill** button in the top right to open the creation dialog (specify Skill Name, auto-generated or custom Slug, and Description).
3. **[2] Search and Filter Skills (*Search & Status filter*):** Type a query into the *Search by name, slug or description...* box, or select a status from *Filter Status* (*All*, *Active*, or *Deleted*).

4. **[3] Inspect Skills Data Table (*Skills Table*):** Review catalog rows displaying *ID*, *Name*, *Slug*, *Status* badge (**Active** or **Deleted**), *Jobs using it* tally, and *Created* date.
5. **[4] Perform Row Actions (*Actions column*):**
    - Click the pencil icon (**Pencil icon**) to edit skill attributes.
    - Click the trash icon (**Trash icon**) to execute a **Soft Delete**. For previously deleted skills (indicated by a red **Deleted** badge), this button transforms into a circular restore icon (**Restore icon**), allowing the administrator to reactivate the skill back to **Active** status.
    - Click the eye icon (**Eye icon**) to view detailed skill metrics at `/admin/skills/[id]`.

Expected Output & Exception Handling:

- **Success Scenario:** Newly added or updated skills become available immediately in the auto-complete suggestions for client job postings and freelancer profiles. Soft deletion sets `deletedAt = NOW()` without breaking existing foreign key associations.
- **Exception Scenarios:** If a user attempts to create a skill with a duplicate Name or Slug, the server rejects the request with an explicit uniqueness validation error.

3.4.2 M15.02 – Job Categories & Industry Classification Management (UC-12.01 to UC-12.06)

- **Screen Name & Route Path:** Job Category Management Console – `/admin/job-categories`.
- **Target Actor:** Administrator.
- **Preconditions:** Administrator has permissions to access `ADMIN_JOB_CATEGORIES`.

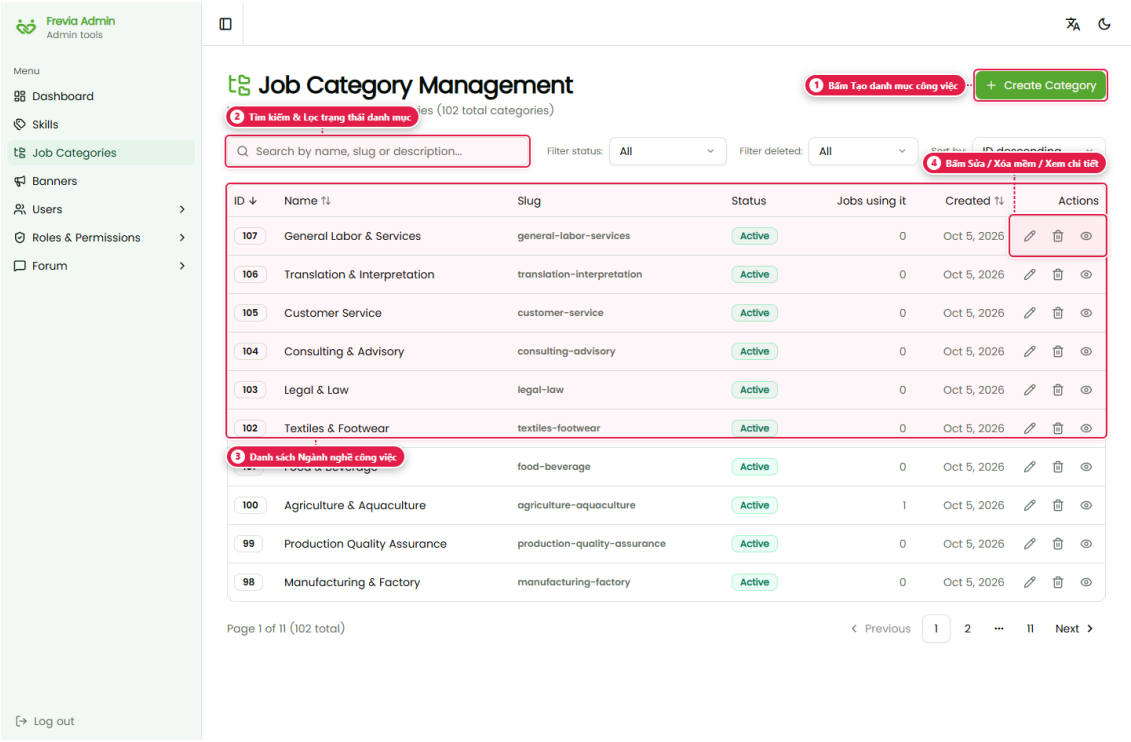


Figure 3.17: Job Categories management console with category hierarchy at `/admin/job-categories`

Step-by-Step User Instructions:

1. In the Admin sidebar, click **Job Categories**.
2. **[1] Create New Job Category (*+ Create Category*):** Click **+ Create Category** to open the category creation dialog (specify Category Name, Slug, Description, and Icon).
3. **[2] Search and Status Filter (*Search & Filter*):** Filter categories by keyword or lifecycle state (*All*, *Active*, *Deleted*).
4. **[3] Inspect Job Categories Table (*Categories Table*):** Examine records displaying *ID*, *Category Name*, *Slug*, *Description*, *Status*, and associated *Jobs Count*.
5. **[4] Category Actions (*Actions column*):** Click the pencil icon to update category details, or click the trash/restore icon to perform soft-delete or restore operations.

Expected Output & Exception Handling:

- **Success Scenario:** Updated job categories immediately update the marketplace search facets and job creation forms.

3.4.3 M15.03 – Forum Discussion Categories Management (UC-13.01 to UC-13.06)

- **Screen Name & Route Path:** Forum Category Management Console – `/admin/categories`.
- **Target Actor:** Administrator.
- **Preconditions:** Administrator has permissions to access `ADMIN_FORUM_CATEGORIES`.

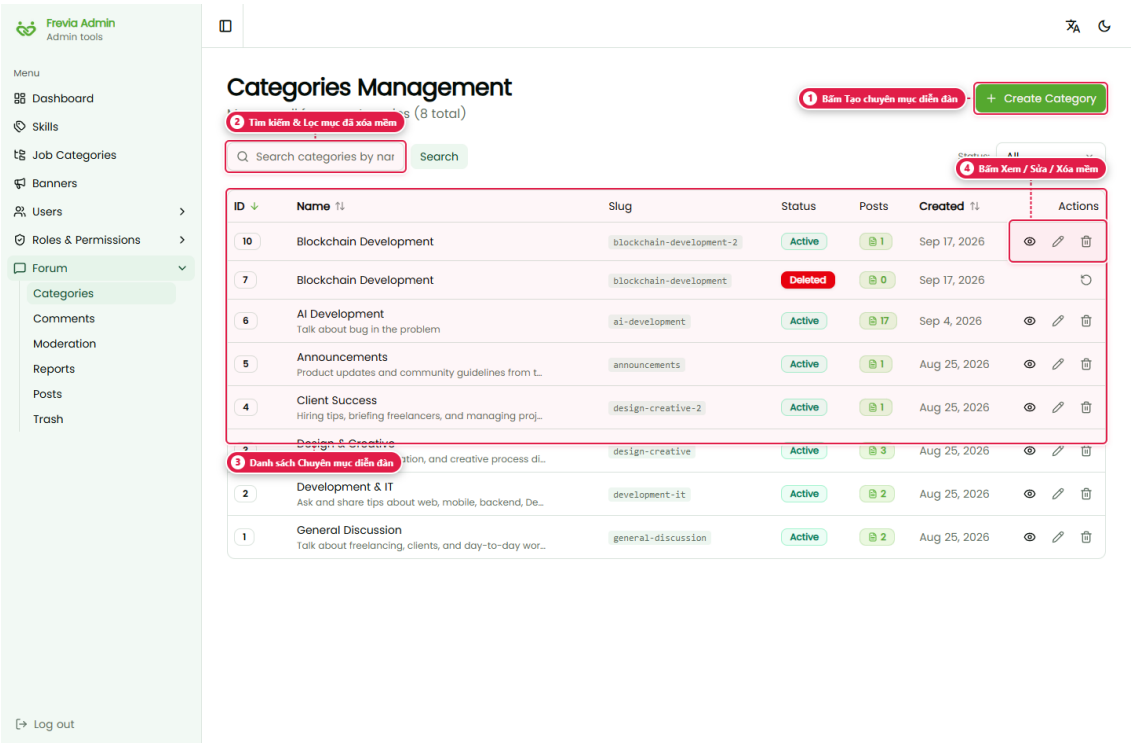


Figure 3.18: Forum Discussion Categories management console at `/admin/categories`

Step-by-Step User Instructions:

1. In the Admin sidebar, navigate to **Forum** → **Categories** (`/admin/categories`).
2. **[1] Create New Forum Category (*+ Create Category*):** Click **+ Create Category** to open the creation dialog (enter Category Name, Slug, Description, and display order).
3. **[2] Search Forum Categories (*Search input*):** Filter community boards by keyword in real time.
4. **[3] Inspect Forum Boards Table (*Forum Categories Table*):** View category attributes including *ID*, *Category Name*, *Slug*, *Topics / Posts Count*, and *Status*.
5. **[4] Manage Forum Categories (*Actions column*):** Use the action icons to edit category details, archive inactive boards via soft-delete, or restore previously archived boards.

Expected Output & Exception Handling:

- **Success Scenario:** Community discussion boards on `/forum` update in real time based on active category configurations.